

Unifying Depth and Width Pruning for LLMs via Binary Knapsack Optimization

Palaash Goel¹ Ayan Sengupta² Akshay Nambi³ Tanmoy Chakraborty^{1,2}

¹Yardi School of Artificial Intelligence, Indian Institute of Technology Delhi

²Department of Electrical Engineering, Indian Institute of Technology Delhi

³Microsoft Research (MSR) India

goelpalaash@scai.iitd.ac.in, ayan.sengupta007@gmail.com,
akshay.nambi@microsoft.com, tanchak@iitd.ac.in

github.com/parmanu-lcs2 parmanu.lcs2.in

Abstract Structured pruning is a promising approach for compressing large language models (LLMs), yet existing methods rely heavily on greedy heuristics that produce myopic decisions, and often fail to precisely meet target compression budgets. We present SNIPER, a two-stage structured pruning framework that solves a knapsack optimization over coarse-granularity components to yield conditionally optimal parameter allocations with respect to fixed importance estimates, followed by a fine-grained pruning stage to meet strict budget constraints. We introduce the Compression Ratio Adherence Factor (CRAFT) to quantify budget fidelity, showing that while existing pruners deviate from target compression ratios by up to 33%, SNIPER achieves near-exact adherence with a CRAFT score of 0.98. Evaluations across four diverse architectures over a set of 18 tasks spanning five domains demonstrate SNIPER’s consistent improvements in average performance retention and task-level stability over six state-of-the-art pruners. Across all pruning configurations, SNIPER achieves an excellent mean rank of 1.25, indicating its robust cross-architectural generalizability and excellent reliability.

Keywords *LLM compression, structured pruning, LLM efficiency, knapsack optimization, dual-axis pruning*

1 Introduction

Large language models (LLMs) have become the foundation of modern natural language processing, driving advances in reasoning, generation, and comprehension across a wide range of tasks [1–4]. These capabilities, however, come at the cost of rapidly increasing parameter counts, posing significant challenges for deployment on resource-constrained hardware such as edge devices and on-device accelerators [5, 6]. While techniques such as quantization [7–9] and knowledge distillation [10, 11] reduce memory footprint or training cost, they largely preserve the original model architecture and therefore do not yield proportional inference speedups. In contrast, structured model pruning directly removes parameter groups, offering a principled route to practical acceleration.

Despite their appeal, existing structured pruning methods suffer from fundamental limitations arising from the interplay between pruning granularity and greedy optimization. Depth-based pruners [12–15] operate on coarse atomic units and often fail to adhere to target compression ratios, while width-based methods [16–18] induce sparsity-driven irregularities in weight tensor shapes, leading to minimal inference speedup [19, 20]. Additionally, existing methods tend to rely on local and greedy heuristics that ignore inter-component dependencies across layers and may discard components that appear weak individually but are globally important. These design choices lead to three recurring failure modes: (i) calibration-induced distributional bias [21] and high task-wise deviations outside the calibration distribution (Figure 1b shows deviation grows to 30%); (ii) irreversible local pruning decisions that cannot guarantee optimal architectural configurations, even with respect to their own importance estimates; and (iii) unreliable adherence to target compression ratios, undermining deployment on memory-constrained systems (Figure 1a highlights deviations upto 33%).

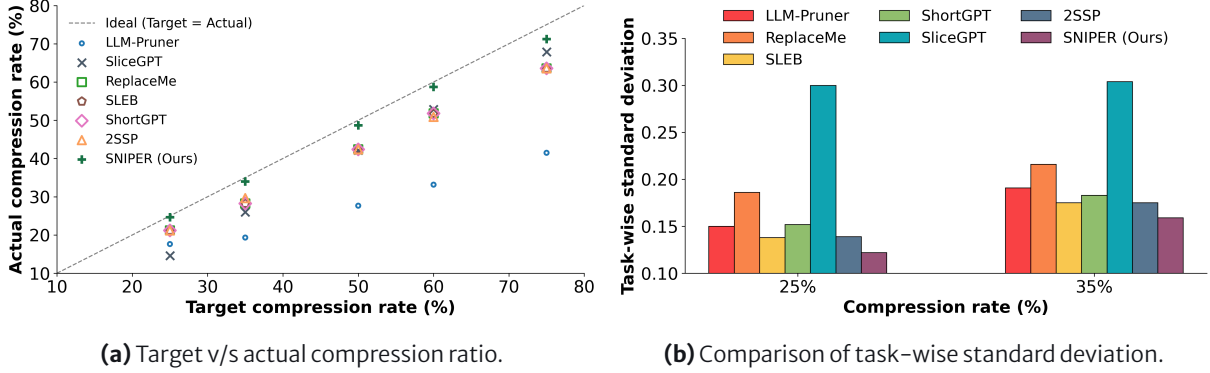


Figure 1. (a) Existing structured pruners exhibit severe “capacity slack”, with the actual compression ratio deviating from the target by up to 33%. SNIPER achieves near-exact budget adherence with a CRAFT of 0.98, compared to the baseline average of 0.78. Quantitative results are provided in Table 8 of Appendix C.1. (b) SNIPER reduces task-wise standard deviation by up to 60% relative to prior methods, mitigating the instability induced by existing pruning strategies.

We address these limitations in this work and summarize our contributions as follows:

- We propose **SNIPER (Structured Knapsack-optimization-based Pruner)**, a novel dual-axis structured pruning framework for LLMs. SNIPER first compresses along the depth-axis by solving a 0/1 knapsack problem over coarse-grained components, then applies a width-pruning phase that lets the model hit the target compression ratio near-perfectly.
- Unlike existing systems that rely on greedy and myopic pruning decisions, SNIPER utilizes dynamic programming to guarantee a conditionally optimal selection of coarse-grained components while pruning along the depth-axis.
- We introduce the Compression Ratio Adherence Factor (CRAFT) to quantify budget fidelity, and show that SNIPER achieves near-exact compression targets, whereas existing structured pruners suffer from substantial capacity slack, thereby undermining utility.
- We construct a comprehensive evaluation suite spanning six modern baselines tested across 18 tasks, five domains, and four diverse architectures – including dense, reasoning-specialized, fused-MLP, and mixture-of-experts models. SNIPER consistently achieves state-of-the-art performance retention, and lower task-level variance under multiple compression regimes.

2 Related Work

The escalating computational demands of Large Language Models (LLMs) have catalyzed extensive research into efficient deployment strategies. Existing approaches generally fall into the categories of quantization [7, 9, 22–24], knowledge distillation [25–27], and activation sparsity [28–31]. While effective, these methods typically preserve the original model architecture. In contrast, *model pruning* explicitly removes parameters to achieve tangible inference speedups and memory reductions.

2.1 Unstructured Model Pruning

Early research predominantly focused on *unstructured* or *semi-structured* sparsity, identifying individual weights for removal based on magnitude [32, 33] or second-order gradient information [34–36]. Although methods like SparseGPT [35] and Wanda [33] achieve high sparsity with minimal perplexity degradation, they typically produce irregular sparse matrices. Consequently, specialized hardware accelerators or N:M sparsity support are often required to translate theoretical FLOPs reductions into actual latency gains [37].

2.2 Structured Pruning for LLMs

To circumvent the hardware dependencies of unstructured sparsity, recent work has shifted towards *structured pruning*, which removes coherent architectural units. We categorize these approaches based on their granularity:

Width Pruning. These methods prune along the channel or embedding dimension, effectively narrowing the model’s matrices. Techniques range from pruning individual attention heads [38–40] to removing entire neurons in MLPs [17]. LLM-Pruner [16] advances this by constructing a dependency graph to ensure structurally coupled parameters are pruned simultaneously. Similarly, SliceGPT [18] projects weight matrices into a lower-dimensional space via PCA, effectively slicing rows and columns. Other decomposition-based methods, such as FWSVD [41], ASVD [42], and SVD-LLM [43], adopt greedy strategies to select and drop redundant neuron blocks from MLP and self-attention modules. While effective for fine-grained compression, width pruning often struggles to fully excise large, redundant computational blocks.

Depth Pruning. Operating at a coarser granularity, depth pruning removes entire transformer layers [15, 44, 45]. Approaches such as ShortGPT [13] and SLEB [14] identify and discard redundant layers using block influence metrics. ReplaceMe [12] mitigates the representation collapse caused by layer removal by substituting contiguous blocks with a learned linear projection. Dynamic schemes, such as PuDDing [46], learn input-conditioned routing policies for selective layer skipping at inference time. Recent hybrid approaches [45, 47] attempt to jointly prune across neurons, heads, and layers, guided by learned or internal importance signals.

2.3 Limitations of Prior Work

Most aforementioned structured approaches share a critical limitation: reliance on *greedy* selection criteria. Decisions are typically made locally (e.g., layer-by-layer) without an optimality-centric view of the parameter-performance trade-off. Furthermore, methods tend to not adhere to the required compression budget, introducing significant “capacity slack”, with deviations of up to 33% from the target budget (Figure 1a). SNIPER addresses these gaps by pruning along two axes: first along the depth wherein component selection is driven by a *conditionally optimal* knapsack optimization algorithm, followed by a width-pruning stage that ensures that the target compression budget is met with high accuracy.

3 Methodology

In this section, we provide a detailed account of the dual-stage pruning mechanism utilized by SNIPER, as illustrated in Figure 2.

3.1 Problem Formulation

Consider an LLM $\mathcal{M}$ consisting of a sequence of L transformer layers. We decompose the model into a set of discrete, prunable components $\mathcal{X}$. Specifically, we adopt a *mixed-granularity* decomposition wherein each transformer layer l is decomposed into its constituent sub-blocks: the Multi-Head Attention block (A_l) and the Feed-Forward Network block (M_l). Thus, the universe of items is defined as: $\mathcal{X} = \bigcup_{l=1}^L \{A_l, M_l\} \cup \{T_l\}$, where T_l represents the atomic retention of the *entire* layer l . Since retaining T_l effectively implies retaining both A_l and M_l , it follows that for any layer l , the choice is mutually exclusive between selecting the composite item T_l or a subset of its components $\{A_l, M_l\}$. Each component $x \in \mathcal{X}$ is associated with a weight $w(x) \in \mathbb{Z}^+$, representing its parameter count, and a value $v(x) \in \mathbb{R}$, representing its contribution to model performance (Section 3.3). Given a target parameter budget C , our objective is to identify the subset $\mathcal{S}^* \subseteq \mathcal{X}$ that maximizes total importance while satisfying the capacity constraint:

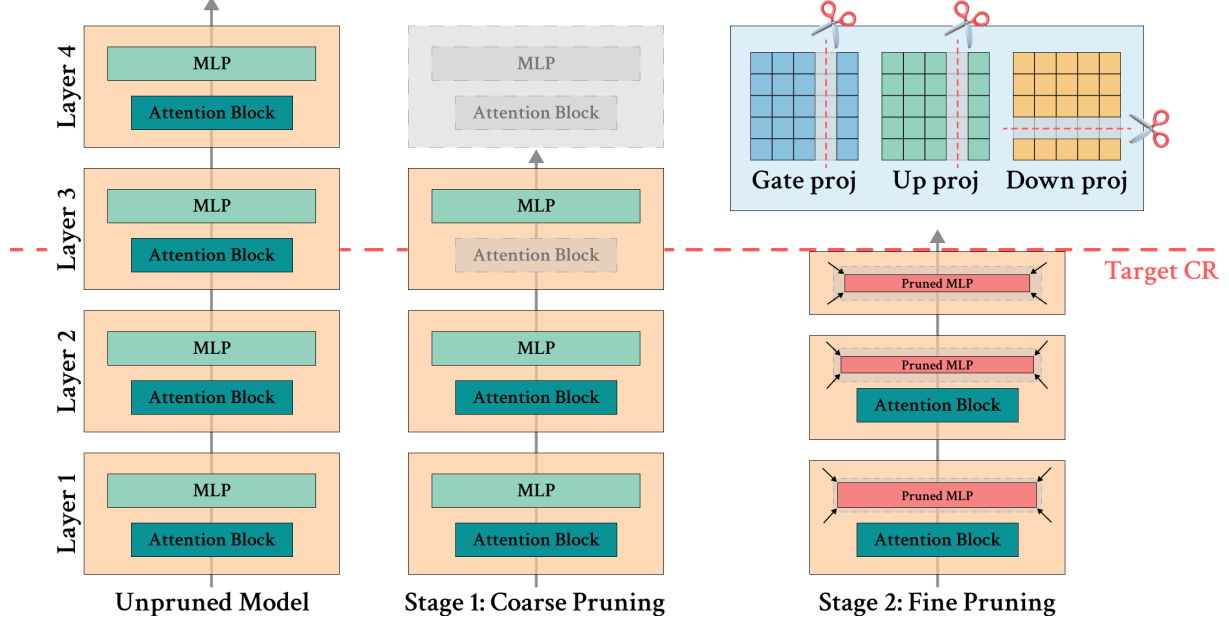


Figure 2. A schematic overview of SNIPER which performs structured pruning in two stages under a strict target compression ratio (CR). **Stage 1: Coarse pruning** removes entire redundant components such as full transformer layers or attention blocks, enabling notable efficiency gains while preserving architectural coherence. **Stage 2: Fine pruning** then operates within the remaining layers, selectively pruning the internal dimensions of the model’s MLPs to precisely meet the target CR. This dual-axis strategy allows SNIPER to combine the efficiency of depth pruning with the precision of width pruning, eliminating capacity slack and avoiding greedy, layer-wise decisions.

$$\mathcal{S}^* = \arg \max_{\mathcal{S} \subseteq \mathcal{X}} \sum_{x \in \mathcal{S}} v(x) \quad \text{s.t.} \quad \sum_{x \in \mathcal{S}} w(x) \leq C \quad (1)$$

This formulation maps the coarse-grained pruning stage directly to the 0/1 Knapsack Problem [48].

3.2 Stage 1: Coarse-Grained Pruning via Dynamic Programming

To solve the optimization problem in Equation 1, we employ a dynamic programming approach. Let the components be indexed sequentially (due to the inherent hierarchy in a typical transformer model, the components can be deterministically sequenced) $i = 1, \dots, N$, where $N = |\mathcal{X}|$. We define the state function $f(i, j)$ as the maximum importance value achievable using a subset of the first i items subject to a capacity limit j . The recurrence relation is defined as:

$$f(i, j) = \begin{cases} f(i-1, j) & \text{if } w(x_i) > j \\ \max(f(i-1, j), \Delta) & \text{if } w(x_i) \leq j \end{cases} \quad (2)$$

where, $\Delta = f(i-1, j - w(x_i)) + v(x_i)$. The first case corresponds to excluding item x_i due to capacity violation, while the second case evaluates the trade-off between exclusion and inclusion. To enforce the mutual exclusivity constraint (i.e., one cannot select both T_l and A_l), we group mutually exclusive items and process them as a single decision step with multiple branches.

Notably, while this algorithm originally runs in $\Theta(N \cdot C)$ time which is intractable when C is in the order of billions, we improve its tractability by discretizing all parameter counts by a discretizing factor, α . This reduces the algorithm’s time complexity to $\Theta(N \lfloor \frac{C}{\alpha} \rfloor)$. As expected, increasing α makes the algorithm faster at the cost of assigning less precise (more coarsely rounded) parameter-count values to each component, leading to a suboptimal knapsack solution or a final compression ratio that deviates further from the target. We provide a detailed analysis of

the same in Appendix 6.5.

Upon computing the terminal state $f(N, \lfloor \frac{C}{\alpha} \rfloor)$, we backtrack to recover the optimal set $\mathcal{S}^*$. We note that $\mathcal{S}^*$ does not represent a global optimum over all possible pruning configurations but rather a *conditional optimum* with respect to fixed component-level importance estimates. Despite being a weaker guarantee of optimality, it surpasses what existing greedy methods provide since they do not offer any optimality guarantee whatsoever, conditional or otherwise.

3.3 Iterative Importance Estimation

A critical challenge in Knapsack-based pruning is assigning static importance values $v(x_i)$ to components that interact non-linearly. To address this, we propose an iterative importance estimation scheme based on marginal contribution. Let $\mathcal{M}^{(i-1)}$ denote the model pruned up to component $i - 1$. We define the importance of component x_i as the divergence induced by removing it from $\mathcal{M}^{(i-1)}$. Let $\mathbf{Z} \in \mathbb{R}^{B \times V}$ be the logits produced by the full model on a calibration batch $\mathbf{X}$, where B and V are the batch size and vocabulary size of the model, respectively. For the i -th component, we evaluate two scenarios:

1. **Retention:** The component is kept. Let $\mathbf{Z}_{\text{retain}}^{(i)}$ be the logits of the model where x_i is present.
2. **Drop:** The component is pruned. Let $\mathbf{Z}_{\text{drop}}^{(i)}$ be the logits of the model where x_i is removed.

The importance $v(x_i)$ is quantified as the marginal degradation in prediction fidelity:

$$v(x_i) = \|\mathbf{Z} - \mathbf{Z}_{\text{drop}}^{(i)}\|_2^2 - \|\mathbf{Z} - \mathbf{Z}_{\text{retain}}^{(i)}\|_2^2 \quad (3)$$

Unlike greedy parameter importance estimation metrics that capture the importance of individual components in isolation, Equation 3 ensures that the importance scores reflect the effect of jointly pruning multiple components, thereby making the scoring function more reliable and representative. We note that SNIPER is similar to existing methods in that it utilizes heuristic proxies for component importance. However, it greatly improves upon these baselines by not making pruning decisions greedily and instead, computing *provably optimal* compression configurations with respect to these importance scores.

3.4 Stage 2: Fine-Grained Residual Pruning

The discrete nature of components in Stage 1 often leaves a residual capacity $\Delta C > 0$. To fully utilize the budget C , we introduce a second stage of fine-grained structured pruning targeting the MLPs within the transformer blocks.

Budget Allocation. We distribute the fine-grained pruning budget N_{total} (total MLP columns to remove) across layers inversely proportional to their coarse importance. Let $\mathbf{u} \in \mathbb{R}^L$ be the vector of layer importance scores where $u_l = v(T_l)$. We define the layer-wise pruning ratio ρ_l via a softmax over negated importance:

$$\rho_l = \frac{\exp(-u_l/\tau)}{\sum_{k=1}^L \exp(-u_k/\tau)} \quad (4)$$

where τ (defaulted to 1) is a temperature parameter. The number of columns to prune from layer l is computed as $N_l = \lfloor \rho_l \cdot N_{\text{total}} \rfloor$.

Column Selection. Modern LLMs [1, 49] utilize variants of the Gated Linear Unit (GLU), where the MLP computation is defined as:

$$\text{MLP}(\mathbf{X}) = (\sigma(\mathbf{X}\mathbf{G}^T) \odot (\mathbf{X}\mathbf{U}^T))\mathbf{D}^T \quad (5)$$

where $\mathbf{U}, \mathbf{G} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ are the up- and gate-projections, respectively, and $\mathbf{D} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ is the down-projection. Here, d_{model} refers to the model’s hidden dimension and d_{ff} is the higher dimension to which embeddings are

projected within its MLPs. To maintain structural consistency, pruning a column index j requires removing the j -th column of $\mathbf{U}$ and $\mathbf{G}$, and the j -th row of $\mathbf{D}$. We define the sensitivity of the j -th neuron index as the magnitude of its contribution to the output:

$$\Omega_j = |\mathbf{D}_{j,:} \cdot (\mathbf{G}_{:,j} \odot \mathbf{U}_{:,j})| \quad (6)$$

Indices with the lowest Ω_j scores are pruned until N_l columns are removed. This ensures that the residual capacity is filled with the least salient parameters, maxing out the available budget.

By integrating the conditionally optimal configurations computed by the Knapsack solver with the precision of fine-grained residual pruning, SNIPER demonstrates high retention capabilities with respect to the unpruned model, while also ensuring high-fidelity adherence to the parameter budget, as reflected by its excellent CRAFTs (Appendix C.1).

4 Experimental Setup

Pruned models. To evaluate architectural generality, we benchmark SNIPER on a diverse set of modern LLMs¹ with different architectural paradigms. These include the instruction-tuned LLaMA-3.1-8B-Instruct [1], the reasoning-oriented Qwen3-8B [2], Phi-4 [50], which employs fused MLP projections, and the Mixture-of-Experts model GPT-OSS-20B [4].

Baselines. We compare SNIPER against six contemporary structured pruning baselines. Width-based methods include SliceGPT [18] and LLM-Pruner [16], while depth-based methods include ReplaceMe [12], SLEB [14], and ShortGPT [13]. We also include 2SSP [51] since it mirrors SNIPER via its dual-axis approach, allowing us to evaluate SNIPER’s effectiveness with respect to analogous pruning approaches. Several baselines are re-engineered to support recent LLM architectures². We note that vector-space width pruners such as SliceGPT are inherently incompatible with expert routing layers and therefore cannot be applied to GPT-OSS. In order to ensure a fair evaluation, we adjust the compression ratios of all methods to yield pruned models of comparable sizes. We provide all implementation details in Appendix A.

Evaluation task suite. Many pruning evaluations rely on a narrow set of tasks, which can obscure degradation in a model’s linguistic capabilities, reasoning, or alignment. For a more comprehensive assessment, we evaluate all models on a testbed of 18 tasks spanning five domains. Generative performance is measured using log-perplexity on WikiText-2 [52] and LAMBADA [53]. World understanding is evaluated using PIQA [54], PROST [55], and CommonsenseQA [56]. Domain-specific knowledge covers STEM reasoning tasks including ARC-Easy and ARC-Challenge [57], mathematical reasoning tasks such as MathQA [58] and OpenBookQA [59], and medical question answering using MedQA [60]. Natural language understanding is evaluated using BLiMP [61], BoolQ [62], Winogrande [63], and CoQA [64]. To assess alignment preservation, we additionally evaluate safety and ethics using TruthfulQA [65], Winogender [66], and Moral Stories [67]. A detailed description of each task is provided in Appendix B.

5 Results

Table 1 compares SNIPER against state-of-the-art structured pruning methods on LLaMA-3.1-8B-Instruct and Qwen3-8B, under two compression ratios (25% and 35%), both with and without recovery fine-tuning (RFT). Across *all configurations except one*, SNIPER achieves the highest average retention performance (Avg RP) while maintaining strong performance across different task groups. While it is marginally outperformed by ReplaceMe and ShortGPT on Llama-3.1-8B-Instruct (25% compression with RFT), SNIPER maintains a near-perfect mean rank of 1.25 across

¹All pretrained checkpoints are obtained from the HuggingFace model hub.

²For instance, ReplaceMe and ShortGPT do not natively support newer LLM variants.

all pruning configurations, yielding competitive performance even at its worst rank of merely 3 (Tables 1 and 2). In contrast, every baseline has at least one configuration in which its performance deteriorates drastically. Unlike its baselines, SNIPER *never* disintegrates. This indicates SNIPER’s unparalleled robustness and generalization capabilities across different architectures and compression regimes. We provide detailed per-task results in Appendix C.2.

Model	CR	Method	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
LLaMA-3.1-8B-Instruct	-	Base	1.69	66.55	56.28	78.52	55.56	-	-
	25%	ReplaceMe	4.91 (2.36)	59.30 (58.31)	40.78 (43.35)	54.97 (75.80)	55.62 (54.79)	74.86 (88.90)	20.40 (10.50)
		SliceGPT	5.11 (3.60)	40.78 (49.10)	26.87 (30.08)	53.75 (62.49)	50.23 (49.50)	62.06 (72.49)	20.20 (17.30)
		LLM-Pruner	3.96 (2.54)	40.56 (44.37)	32.14 (36.32)	49.85 (62.43)	51.87 (50.99)	64.56 (75.11)	22.40 (17.70)
		SLEB	3.37 (2.25)	42.20 (47.52)	37.06 (38.79)	50.78 (65.98)	51.01 (50.74)	67.91 (78.45)	20.30 (16.80)
		ShortGPT	13.20 (2.46)	48.70 (59.22)	31.68 (42.42)	34.50 (74.77)	51.01 (55.34)	57.50 (88.47)	26.10 (11.20)
		2SSP	2.77 (2.13)	49.33 (54.23)	36.67 (39.75)	68.14 (71.59)	50.93 (51.42)	76.65 (83.19)	14.00 (13.90)
		SNIPER	3.15 (2.29)	51.81 (57.89)	39.00 (41.71)	64.95 (74.41)	53.02 (53.97)	77.03 (87.34)	12.70 (10.00)
	35%	ReplaceMe	5.86 (3.19)	38.42 (39.81)	29.01 (29.62)	36.09 (50.55)	51.24 (49.58)	56.25 (65.16)	27.10 (22.60)
		SliceGPT	6.44 (4.51)	38.43 (44.77)	25.68 (28.41)	42.72 (55.22)	50.86 (51.23)	56.74 (68.08)	22.50 (18.60)
		LLM-Pruner	4.94 (2.94)	37.18 (38.49)	29.61 (33.17)	44.60 (58.50)	51.33 (50.28)	59.66 (69.93)	23.70 (19.40)
		SLEB	4.47 (2.71)	39.20 (40.57)	30.99 (32.95)	44.13 (57.87)	50.87 (51.29)	60.72 (70.58)	24.10 (20.10)
		ShortGPT	13.14 (2.94)	40.77 (43.64)	28.99 (32.50)	36.52 (69.91)	55.03 (54.67)	56.14 (76.98)	28.00 (18.70)
		2SSP	3.77 (2.45)	37.28 (47.62)	29.39 (35.66)	55.55 (68.03)	50.50 (50.19)	64.21 (77.26)	19.10 (15.80)
		SNIPER	4.70 (2.63)	41.16 (48.52)	32.18 (34.73)	53.52 (68.31)	51.29 (50.95)	64.96 (77.26)	19.50 (14.80)
Qwen3-8B	-	Base	2.01	66.32	58.60	76.20	55.90	-	-
	25%	ReplaceMe	4.37 (2.76)	37.40 (43.98)	30.21 (36.83)	41.44 (55.52)	50.47 (50.66)	59.92 (72.30)	23.80 (18.60)
		SliceGPT	15.72 (10.75)	34.01 (34.48)	24.64 (25.05)	29.46 (29.54)	51.00 (51.35)	48.68 (50.07)	27.50 (30.00)
		LLM-Pruner	3.20 (2.46)	48.73 (53.44)	31.85 (37.28)	63.89 (67.05)	50.99 (52.20)	72.64 (80.11)	15.90 (15.00)
		SLEB	3.75 (2.61)	43.11 (50.41)	36.93 (38.38)	48.84 (64.50)	51.48 (53.12)	67.53 (78.89)	20.20 (13.80)
		ShortGPT	8.98 (3.14)	43.76 (55.87)	33.09 (39.34)	46.98 (67.98)	55.02 (54.14)	63.03 (80.21)	22.90 (15.20)
		2SSP	6.37 (3.11)	34.79 (50.10)	26.31 (33.86)	39.98 (58.74)	52.19 (52.44)	56.09 (72.92)	23.10 (16.30)
		SNIPER	3.65 (2.72)	50.81 (55.93)	36.21 (41.36)	61.04 (69.31)	54.90 (53.70)	74.41 (82.64)	15.00 (12.20)
	35%	ReplaceMe	5.99 (4.20)	37.63 (38.12)	32.30 (28.86)	40.78 (52.27)	51.47 (52.59)	59.40 (64.00)	25.90 (21.60)
		SliceGPT	16.13 (10.82)	33.49 (34.39)	24.78 (24.58)	29.53 (29.87)	50.99 (52.44)	48.67 (50.17)	27.70 (30.40)
		LLM-Pruner	5.25 (3.46)	39.24 (45.25)	26.47 (31.86)	50.05 (55.02)	50.75 (51.39)	60.63 (68.34)	21.90 (19.10)
		SLEB	4.94 (3.10)	38.46 (45.15)	31.95 (34.40)	38.99 (53.56)	51.55 (52.43)	59.67 (70.47)	24.70 (17.50)
		ShortGPT	10.67 (3.98)	45.56 (50.25)	30.96 (34.82)	38.09 (59.54)	54.02 (53.11)	58.69 (72.27)	27.10 (18.30)
		2SSP	7.42 (3.56)	34.71 (46.37)	25.92 (31.94)	36.63 (56.68)	51.57 (51.68)	54.15 (69.23)	24.40 (17.50)
		SNIPER	5.11 (3.21)	44.04 (50.81)	30.43 (35.45)	48.68 (63.35)	52.76 (52.81)	63.71 (75.13)	20.10 (15.90)

Table 1. Comparison of different pruning methods across various compression ratios (CR) without and (with) recovery fine-tuning (RFT). We report the scores for each task group, the average retention performance (RP) (ratio of pruned and base model performances), as well as the standard deviation across tasks for each method. Per-task results are highlighted in Tables 9, 10, 11 and 12 of Appendix C.2.

5.1 Consistency and robustness

Beyond average performance, SNIPER exhibits substantially lower task-level standard deviations (Std RP) across *all configurations*, even including settings where it does not outperform all baselines. For example, at 35% compression on Qwen3-8B with RFT, SNIPER reduces task-wise variance to 15.90%, compared to 17–30% for competing pruners. This behavior highlights SNIPER’s ability to avoid a common failure mode of greedy pruning strategies, which often over-optimize for isolated metrics (e.g., perplexity) at the expense of broader task performance. By combining the benefits of conditional optimality during coarse-grained pruning with that of a budget-filling fine-grained stage, SNIPER is able to maintain superior cross-task consistency across diverse model architectures.

5.2 Results on larger and diverse architectures

Table 3 evaluates SNIPER on larger and architecturally distinct models, i.e, Phi-4-14B and GPT-OSS-20B, under a 35% compression regime with RFT. Despite these departures from standard dense transformers, SNIPER consistently

Method	Mean Rank (↓)	Best Rank (↓)	Worst Rank (↓)
ReplaceMe	4.75	1	7
SliceGPT	6.50	5	7
LLM-Pruner	4.00	2	6
SLEB	3.63	3	5
ShortGPT	3.70	2	7
2SSP	3.55	1	6
SNIPER	1.25	1	3

Table 2. Ranking different pruning methods across all available configurations.

Model	CR	Method	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
Phi-4-14B	-	Base	1.65	70.11	56.73	79.89	59.46	-	-
	35%	ShortGPT	3.35	54.18	40.05	65.39	57.44	77.18	16.60
		2SSP	3.07	47.61	33.35	62.66	51.62	70.16	16.20
		SNIPER	2.55	58.07	44.14	70.10	53.70	81.58	12.60
GPT-OSS-20B	-	Base	2.22	65.65	56.07	75.37	58.16	-	-
	35%	ShortGPT	3.06	46.18	38.03	45.16	52.24	70.51	22.10
		2SSP	2.99	56.16	43.19	68.99	56.38	84.68	11.40
		SNIPER	2.71	55.62	45.28	69.94	55.47	86.99	8.80

Table 3. Performance analysis on larger and architecturally diverse models – Phi-4 and GPT-OSS-20B with RFT. Detailed results provided in Tables 13 and 14 of Appendix C.2.

attains the highest Avg RP while maintaining the lowest Std RP. On Phi-4-14B, SNIPER attains an Avg RP of 81.58%, substantially outperforming ShortGPT and 2SSP by 4.4% and 11.42%, respectively. SNIPER demonstrates similar superiority on GPT-OSS-20B, achieving an Avg RP of 86.99% - 2.31% higher than 2SSP and 16.48% better than ShortGPT. Notably, on GPT-OSS-20B, 2SSP operates solely via its first stage due to how it distributes the pruning budget between its two stages, effectively turning its second stage into a no-op. In contrast, SNIPER ensures that it leverages both of its stages to yield a superior compressed model.

5.3 Adherence to Compression Ratio

We analyze the effect of fixing the target compression ratio for each method and observing its adherence to the same. We observe that existing methods demonstrate significant deviations of upto 33% from the target ratio, leading to severe under-pruning and an erosion of trust in the pruning process (Figure 1 and Appendix C.1). In contrast, SNIPER exhibits remarkable reliability by adhering strictly to these targets with a near-ideal CRAFT of 0.98 across a wide range of budgets. Unlike its baselines, SNIPER eliminates reliance on hit-and-trial strategies to obtain a model of a specific size, leading to significant practical advantages.

5.4 Ablation on SNIPER

We provide additional ablation experiments in Table 4 and Figure 3 and analyze the effectiveness of various components and design choices of SNIPER.

Dual-Axis Pruning. We ablate SNIPER’s coarse- and fine-grained components, measuring Avg RP and Std RP (Figure 3). Coarse pruning alone reaches a strong Avg RP of 97.47%, but its high Std RP (10.50%) reflects a key limitation: operating on entire structural groups rather than individual neurons causes some important neurons to inevitably be pruned, destabilizing downstream performance. Its structured removals, however, preserve weight tensor

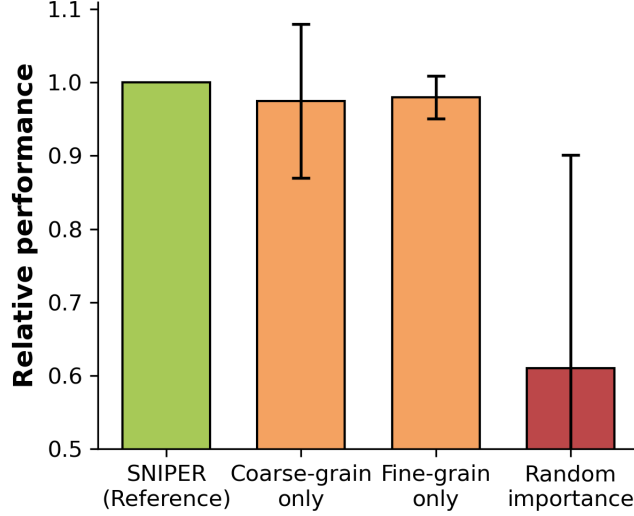


Figure 3. Ablation study of SNIPER showing the effect of coarse and fine pruning stages. Combining both stages yields the best performance and stability, while removing either component leads to lower performance retention or instability.

Variant	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
SNIPER-MAG	3.02	54.76	39.61	64.48	52.49	78.87	13.80
SNIPER-ACT	2.74	53.81	40.27	65.90	54.23	80.87	13.80
SNIPER-LeaveOneOut	4.08	37.33	29.11	47.93	51.44	62.01	23.30
SNIPER-Uniform	2.77	52.40	41.19	68.22	53.51	81.26	13.70
SNIPER	2.72	55.93	41.36	69.31	53.70	82.64	12.20

Table 4. Additional ablation results for SNIPER obtained after compressing Qwen3-8B by 25% (with RFT).

shapes, making it the primary driver of inference speedup. The fine-grained variant attains a marginally better Avg RP (97.95%) with far lower Std RP (2.90%), confirming that precise, neuron-level interventions better preserve cross-task consistency. However, it produces irregularly-shaped weight tensors, yielding little inference speedup. The two stages are thus complementary: coarse pruning delivers hardware-friendly speedups at the cost of consistency, while fine-grained pruning restores consistency but lacks efficiency gains due to induced irregularities. Additionally, random importance assignment yields a low Avg RP (63.24%) and high Std RP (26.90%), confirming the necessity of principled importance estimation.

Column Selection Heuristic. We analyze the efficacy of our column selection heuristic (Equation 6) by replacing it with two popular alternatives: (a) magnitude-based [32] and (b) activation-aware [33] pruning, corresponding to the SNIPER-MAG and SNIPER-ACT variants in Table 4, respectively. Both variants perform approximately 2–4% worse than SNIPER on average and exhibit 13% higher per-task deviation, demonstrating the effectiveness of our strategy at maintaining strong and stable performance.

Iterative Pruning for Importance Estimation. To estimate component importance, SNIPER computes the marginal degradation in performance incurred by dropping a component from a partially pruned model, whose pruning configuration is determined by our dynamic programming solver (Section 3.3). We compare this against the leave-one-out strategy employed by baselines such as SLEB [14], denoted SNIPER-LeaveOneOut. This variant incurs a significant drop of 20% in average performance and nearly twice the task-wise standard deviation of SNIPER, demonstrating that estimating component importance on iteratively optimal pruning configurations captures global inter-component

dependencies—something leave-one-out strategies fail to model due to their limited scope.

Fine-Grained Pruning Budget Distribution. To determine how many columns to drop from each MLP in Stage 2 (Section 3.4), SNIPER uses the component importance scores from Section 3.3 (Equation 3) to distribute the pruning budget inversely proportional to importance (Equation 4), pruning redundant MLPs more aggressively while compressing salient ones more conservatively. We evaluate the SNIPER-Uniform variant, which distributes the fine-grained pruning budget uniformly across all MLPs. As shown in Table 4, this variant not only incurs a 1.5% drop in average performance but, more critically, exhibits 12.3% higher task-wise standard deviation, indicating uneven performance retention across domains.

6 Discussion

# Calibration Samples	Method	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
50	ShortGPT	3.14	55.87	39.34	67.98	54.14	80.21	15.20
	2SSP	3.11	50.10	33.86	58.74	52.44	72.92	16.30
	SNIPER	2.72	55.93	41.36	69.31	53.70	82.64	12.20
250	ShortGPT	3.10	58.02	39.95	67.66	53.90	80.98	13.30
	2SSP	3.02	51.68	33.62	63.59	53.30	75.31	15.10
	SNIPER	2.76	54.34	40.21	68.25	53.58	81.26	12.40
1000	ShortGPT	3.10	58.60	39.96	67.86	54.09	81.21	14.00
	2SSP	3.00	52.98	33.90	60.05	52.20	74.26	15.40
	SNIPER	2.75	53.79	40.08	69.30	53.93	81.50	12.90

(a) Calibration data sample size with Slim Orca [68].

Calibration Dataset	Method	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
Slim Orca [68]	ShortGPT	3.14	55.87	39.34	67.98	54.14	80.21	15.20
	2SSP	3.11	50.10	33.86	58.74	52.44	72.92	16.30
	SNIPER	2.72	55.93	41.36	69.31	53.70	82.64	12.20
Alpaca [69]	ShortGPT	2.93	58.48	41.19	68.64	53.73	82.59	13.50
	2SSP	2.71	53.44	39.26	63.75	52.84	79.06	14.30
	SNIPER	2.75	54.80	40.49	68.67	53.79	81.79	13.10
C4 [70]	ShortGPT	3.65	61.19	39.28	66.70	54.47	80.63	15.30
	2SSP	3.35	39.65	31.05	55.10	52.19	67.48	18.80
	SNIPER	2.75	54.77	40.66	68.61	54.11	81.93	13.10

(b) Impact of calibration dataset with sample size of 50.

Table 5. Impact of calibration on compressed Qwen3-8B with RFT. Detailed results provided in Tables 15 and 16 of Appendix C.3.

6.1 Impact of calibration on pruning robustness.

We analyze the impact of calibration data on structured pruning by varying both *calibration set size* and *calibration distribution* using Qwen3-8B with post-pruning RFT (Table 5). As the number of calibration samples increases from 50 to 1000, 2SSP demonstrates a marked increase in its Avg RP (72.92% $\rightarrow$ 75.31), followed by a decline in the same (75.31% $\rightarrow$ 74.26%), while its Std RP exhibits the reverse pattern (16.30% $\rightarrow$ 15.10% $\rightarrow$ 15.40%), indicating its relative instability and sensitivity to calibration sample count. While ShortGPT remains comparatively stable in average performance, it consistently suffers from higher variance across task groups. In contrast, SNIPER achieves the best trade-off between performance and consistency across all calibration sizes, reducing task-wise standard deviation by an average of 11.45% relative to ShortGPT. A more noticeable trend is observed when varying calibration

distributions: both ShortGPT and 2SSP show large performance fluctuations across Slim Orca [68], Alpaca [69], and C4 [70], whereas SNIPER maintains consistently high Avg RP and low Std RP. These results indicate that SNIPER’s pruning strategy is significantly less sensitive to calibration choices, further reinforcing its robustness with respect to its baselines. Detailed results are provided in Appendix C.3.

CR	Transfer regime	Generative (Log-PPL ↓)	World (Acc ↑)	Domain (Acc ↑)	NLU & NLI (Acc ↑)	Safety (Score ↑)	Avg RP (%) (↑)	Std RP (%) (↓)
25%	25% → 25% (No transfer)	3.65	50.81	36.21	61.04	54.90	74.41	15.00
	35% → 25% (High-to-low)	3.66	50.59	36.12	60.78	54.89	74.24	17.00
	50% → 25% (High-to-low)	3.63	49.48	35.73	61.58	54.64	73.92	17.40
35%	35% → 35% (No transfer)	5.11	44.04	30.43	48.68	52.76	63.71	20.10
	25% → 35% (Low-to-high)	5.07	42.73	29.29	49.72	53.75	63.48	22.90
	50% → 35% (High-to-low)	5.08	44.42	30.43	47.44	54.15	63.83	22.90
50%	50% → 50% (No transfer)	11.56	36.37	26.80	33.63	53.51	53.30	28.60
	25% → 50% (Low-to-high)	9.55	35.51	25.86	30.69	51.97	51.46	30.00
	35% → 50% (Low-to-high)	11.46	36.80	26.23	31.95	53.71	52.45	30.90

Table 6. Transferability of importance scores across compression ratios. We reuse importance scores computed at one compression ratio to prune the model at a different target ratio (e.g., 35% → 25%), and compare against recomputing scores from scratch (Table 1). Table 17 of Appendix C.4 displays transferability metrics across each task.

6.2 Transferability of importance scores.

Table 6 evaluates the robustness of SNIPER’s importance scores when transferred across compression ratios. Although SNIPER is designed to recompute scores for each target budget, transferring them across ratios (35% → 25% and vice versa) results in only marginal degradation in average retention performance with a modest increase in task-wise variance. For example, the 35% → 25% transfer preserves nearly identical average RP (74.24% vs. 74.41%), with the reverse exhibiting a similarly small drop (63.48% vs. 63.71%). This stability indicates that SNIPER’s importance signals reflect intrinsic structural salience rather than budget-specific artifacts, while the slight variance increase highlights the benefit of recomputing scores for strict optimality. Overall, transferability offers a favorable efficiency–robustness trade-off, allowing users to bypass SNIPER’s most time-intensive step, i.e, importance computation, with minimal performance sacrifice. We provide task-wise results in Appendix C.4.

6.3 Interpretable structural salience across layers.

Figure 4 highlights an important qualitative property of SNIPER: beyond improved performance, it yields an interpretable measure of layer-wise importance. The learned scores exhibit a clear structure — consistently high importance assigned to early and late transformer layers and systematically lower importance in intermediate layers — with pruned layers predominantly drawn from the middle while boundary layers are preserved. Moreover, substantial overlap in evicted layers across compression budgets indicates that importance estimates capture stable architectural salience rather than budget-specific noise. This structured pattern aligns with known functional roles of transformer layers, suggesting that SNIPER provides a principled, human-interpretable signal for understanding redundancy and capacity allocation in LLMs.

6.4 Runtime analysis, latency, and practical utility

Figure 5a compares the end-to-end runtime of representative depth-, width-, and mixed-granularity pruning methods on Qwen3-8B. Greedy depth pruners such as SLEB are the fastest (≈ 23 minutes) as importance estimation is performed over a small number of coarse atomic units, whereas width-based methods such as SliceGPT incur substantially higher runtime (≈ 55 minutes) due to computations over large sets of fine-grained elements. SNIPER incurs a total runtime that is only about 4 minutes higher than depth-only methods, with approximately 96% of runtime

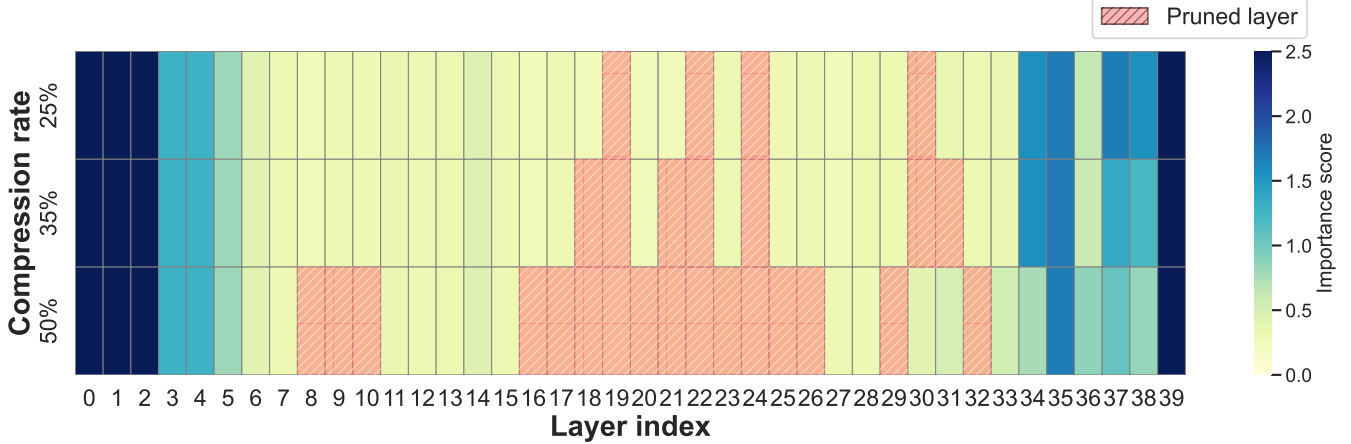


Figure 4. Layer-wise importance scores and pruning decisions for Phi-4-14B across multiple compression ratios. The substantial overlap in pruned layers across compression ratios indicates stable, interpretable importance estimates that reflect intrinsic architectural redundancy rather than budget-specific noise.

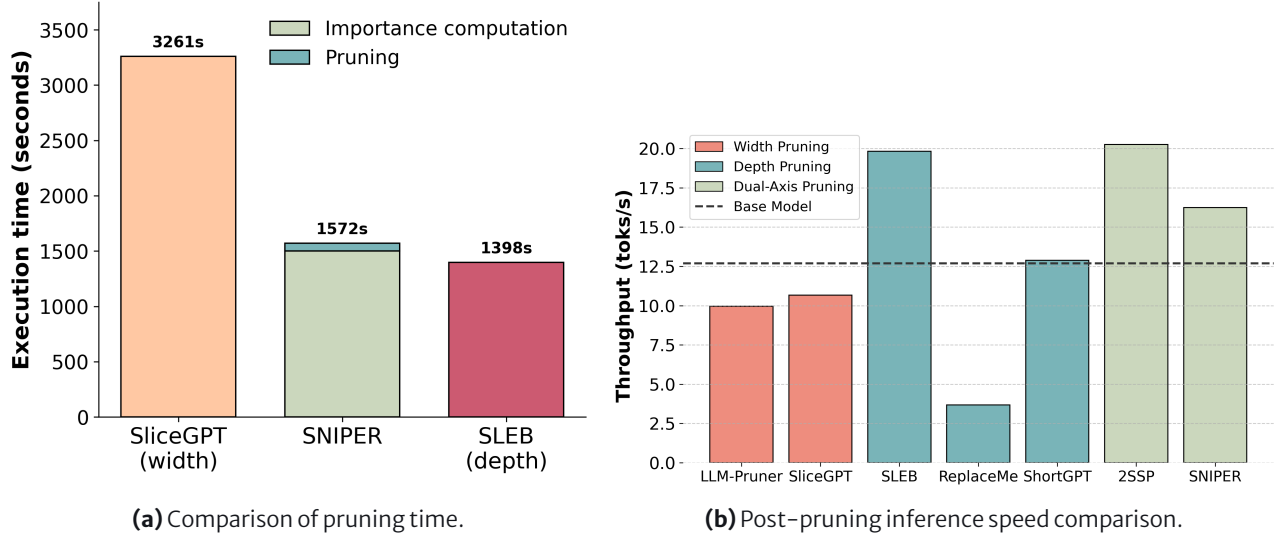


Figure 5. Pruning runtime (a) and post-pruning inference speed (b) for different structured pruning method. All benchmarking conducted on Qwen3-8B model at 25% compression.

devoted to importance estimation while the actual pruning step requires only 56 seconds. This separation enables amortization of the dominant cost across multiple pruning runs by reusing importance scores (Appendix 6.2), while requiring GPU acceleration only during importance estimation. We further analyze post-pruning inference speedup at 25% compression in Figure 5b. Width pruners fall below the base model due to hardware-unfriendly irregular tensor shapes whereas pure depth pruners achieve the largest raw speedups by eliminating entire compute blocks. Furthermore, both dual-axis approaches yield notable latency improvements by offsetting irregular tensor penalties through component elimination. We note that while SNIPER lags behind 2SSP in realized efficiency gains (16.24 vs. 20.26 toks/s), it compensates with consistently strong performance and robustness across architectures and pruning regimes. We hypothesize that 2SSP’s latency gains stem from its treatment of attention blocks as the sole coarse-grained component, whereas SNIPER also considers MLP blocks at this granularity, trading some throughput for more balanced structural coverage.

α	CRAFT	Run-time of DP Solver (in seconds)
8	0.98	22.18
32	0.98	5.84
128	0.98	1.46
512	0.98	0.28
8,192	0.98	0.02
32,768	0.98	0.01
100,000	1.79 (over-pruning)	<0.01

Table 7. Quantitative analysis of the effect of different values of the discretizing factor, α , on CRAFT and algorithm runtime. These results were obtained on Qwen3-8B.

6.5 Analysis of Different Values of Discretizing Factor

As demonstrated in Table 7, SNIPER is robust to a large range of values for α and maintains a near-perfect CRAFT of 0.98 for all values from 8 to 32768. As expected, the runtime decreases as α increases. However, as α reaches exceptionally high values such as 100,000, it causes large components to be discretized into the same bucket as far smaller components, leading to a notable over-shooting of the compression budget. Therefore, to ensure α generalizes well across architectures while keeping runtime low, we select a conservative value of $\alpha = 32$. Notably, for all α in the range 8–32768, the resulting pruned models are identical and therefore yield the same performance.

7 Conclusion

We presented SNIPER, a dual-axis structured pruning framework that compresses along the model’s depth via a knapsack-based objective, guaranteeing a conditionally optimal selection of coarse-grained components and follows that with a width-pruning stage. Through its two stage operation, SNIPER achieves precise budget adherence, strong performance retention, and reduced task-level variance across diverse model architectures. Extensive experiments demonstrate consistent improvements over state-of-the-art structured pruners while performing competitively even under aggressive compression. Beyond performance gains, SNIPER offers a practical deployment advantage by enabling amortized pruning through reusable importance estimates under strict budget constraints. Together, these results underscore the value of non-greedy optimization as a principled foundation for reliable and deployable LLM pruning.

8 Limitations

SNIPER currently employs homogeneous pruning signals across all model components. Incorporating non-homogeneous, expert-specific signals for Mixture-of-Experts (MoE) architectures represents a natural extension that could further enhance pruning efficacy in such settings. Additionally, while SNIPER provides conditional optimality guarantees during component removal, extending theoretical analogous guarantees to other stages of the pipeline such as importance estimation, remains a compelling direction for future work.

9 Ethical Considerations

This work presents SNIPER, a dual-axis structured pruning framework for compressing LLMs. We reflect on the ethical implications of our research below.

Utilization of Public Artifacts. In this work, we employ several publicly available models such as Qwen3-8B [2], LLaMA-3.1-8B-Instruct [1], Phi-4 [50], and GPT-OSS-20B [4]. We also use the publicly available Slim Orca [68]

dataset for pre-compression calibration and post-compression RFT. Furthermore, we utilize the Alpaca [69] and C4 [70] datasets to test the effect of calibration data distribution on the pruning decisions made by each method. We strictly adhere to the terms of use of each of the artifacts used by us.

Intended Use. SNIPER is designed to prune LLMs and make them more accessible to practitioners working in resource-constrained environments. However, we recognize that increasing accessibility also increases the likelihood of a pruned model being misused. Therefore, we implore users of our framework to use compressed models by adhering to the usage policies of the respective base models.

Environmental Impact. As LLMs continue to grow in size, the resources required to run them have reached a concerning point, and will likely continue to grow. However, through model compression, we primarily aim to limit the amount of such required resources, including electricity for powering machines that run LLMs and water for cooling them. We believe that this work will have a positive effect on the environment by making it possible to run more economical variants of resource-intensive LLMs.

Acknowledgment

T. Chakraborty acknowledges the support of the Microsoft Research India Research Grant and the Rajiv Khemani Young Faculty Chair Professorship in Artificial Intelligence.

References

- [1] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnston, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenheide, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute,

- E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A. L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [2] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, "Qwen3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
- [3] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [4] OpenAI, :, S. Agarwal, L. Ahmad, J. Ai, S. Altman, A. Applebaum, E. Arbus, R. K. Arora, Y. Bai, B. Baker, H. Bao, B. Barak, A. Bennett, T. Bertao, N. Brett, E. Brevdo, G. Brockman, S. Bubeck, C. Chang, K. Chen, M. Chen, E. Cheung, A. Clark, D. Cook, M. Dukhan, C. Dvorak, K. Fives, V. Fomenko, T. Garipov, K. Georgiev, M. Glaese, T. Gogineni, A. Goucher, L. Gross, K. G. Guzman, J. Hallman, J. Hehir, J. Heidecke, A. Helyar, H. Hu, R. Huet, J. Huh, S. Jain, Z. Johnson, C. Koch, I. Kofman, D. Kundel, J. Kwon, V. Kyrilov, E. Y. Le, G. Leclerc, J. P. Lennon, S. Lessans, M. Lezcano-Casado, Y. Li, Z. Li, J. Lin, J. Liss, Lily, Liu, J. Liu, K. Lu, C. Lu, Z. Martinovic, L. McCallum, J. McGrath, S. McKinney, A. McLaughlin, S. Mei, S. Mostovoy, T. Mu, G. Myles, A. Neitz, A. Nichol, J. Pachocki, A. Paino, D. Palmie, A. Pantuliano, G. Parascandolo, J. Park, L. Pathak, C. Paz, L. Peran, D. Pimenov, M. Pokrass, E. Proehl, H. Qiu, G. Raila, F. Raso, H. Ren, K. Richardson, D. Robinson, B. Rotsted, H. Salman, S. Sanjeev, M. Schwarzer, D. Sculley, H. Sikchi, K. Simon, K. Singhal, Y. Song, D. Stuckey, Z. Sun, P. Tillet, S. Toizer, F. Tsimpouras, N. Vyas,

- E. Wallace, X. Wang, M. Wang, O. Watkins, K. Weil, A. Wendling, K. Whinnery, C. Whitney, H. Wong, L. Yang, Y. Yang, M. Yasunaga, K. Ying, W. Zaremba, W. Zhan, C. Zhang, B. Zhang, E. Zhang, and S. Zhao, “gpt-oss-120b & gpt-oss-20b model card,” 2025. [Online]. Available: <https://arxiv.org/abs/2508.10925>
- [5] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, “A survey on model compression for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.07633>
- [6] C. V. Nguyen, X. Shen, R. Aponte, Y. Xia, S. Basu, Z. Hu, J. Chen, M. Parmar, S. Kunapuli, J. Barrow, J. Wu, A. Singh, Y. Wang, J. Gu, F. Dernoncourt, N. K. Ahmed, N. Lipka, R. Zhang, X. Chen, T. Yu, S. Kim, H. Deilamsalehy, N. Park, M. Rimer, Z. Zhang, H. Yang, R. A. Rossi, and T. H. Nguyen, “A survey of small language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.20011>
- [7] A. Bhandare, V. Sripathi, D. Karkada, V. Menon, S. Choi, K. Datta, and V. Saletore, “Efficient 8-bit quantization of transformer neural machine language translation model,” 2019. [Online]. Available: <https://arxiv.org/abs/1906.00532>
- [8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: efficient finetuning of quantized llms,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [9] X. Ding, X. Liu, Z. Tu, Y. Zhang, W. Li, J. Hu, H. Chen, Y. Tang, Z. Xiong, B. Yin, and Y. Wang, “CBQ: Cross-block quantization for large language models,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=eW4yh6HKz4>
- [10] Y. Gu, L. Dong, F. Wei, and M. Huang, “Minillm: Knowledge distillation of large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2306.08543>
- [11] A. Sengupta, S. Dixit, M. S. Akhtar, and T. Chakraborty, “A good learner can teach better: Teacher-student collaborative knowledge distillation,” in *Proceedings of the The Twelfth International Conference on Learning Representations, Virtual Event, 2023*, pp. 25–29.
- [12] D. Shopkhoev, A. Ali, M. Zhussip, V. Malykh, S. Lefkimmiatis, N. Komodakis, and S. Zagoruyko, “Replaceme: Network simplification via depth pruning and transformer block linearization,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.02819>
- [13] X. Men, M. Xu, Q. Zhang, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, “Shortgpt: Layers in large language models are more redundant than you expect,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.03853>
- [14] J. Song, K. Oh, T. Kim, H. Kim, Y. Kim, and J.-J. Kim, “SLEB: Streamlining LLMs through redundancy verification and elimination of transformer blocks,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 46 136–46 155. [Online]. Available: <https://proceedings.mlr.press/v235/song24f.html>
- [15] A. Gromov, K. Tirumala, H. Shapourian, P. Gloriosi, and D. Roberts, “The unreasonable ineffectiveness of the deeper layers,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=ngmEcEer8a>
- [16] X. Ma, G. Fang, and X. Wang, “Llm-pruner: on the structural pruning of large language models,” in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [17] A. Sengupta, S. Chaudhary, and T. Chakraborty, “You only prune once: Designing calibration-free model compression with policy learning,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=5RZoYIT3u6>
- [18] S. Ashkboos, M. L. Croci, M. G. do Nascimento, T. Hoefler, and J. Hensman, “Slicegpt: Compress large language models by deleting rows and columns,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.15024>
- [19] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and H.-K. Song, “Shortened llama: Depth pruning for large language models with comparison of retraining methods,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.02834>

- [20] X. Ding, R. Sun, Y. Zhang, X. Yan, Y. Zhou, K. Huang, S. Fu, A. I. Aviles-Rivero, C. Xie, and Y. Zhu, “Sliding-window merging for compacting patch-redundant layers in llms,” *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 25, p. 20826–20834, Mar. 2026. [Online]. Available: <http://dx.doi.org/10.1609/aaai.v40i25.39222>
 - [21] Y. Ji, Y. Xiang, J. Li, Q. Xia, P. Li, X. Duan, Z. Wang, and M. Zhang, “Beware of calibration data for pruning large language models,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=x83w6yGIWb>
 - [22] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-bert: Integer-only bert quantization,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 5506–5518. [Online]. Available: <https://proceedings.mlr.press/v139/kim21d.html>
 - [23] S.-y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, “Llm-fp4: 4-bit floating-point quantized transformers,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2023, p. 592–605. [Online]. Available: <http://dx.doi.org/10.18653/v1/2023.emnlp-main.39>
 - [24] Z. Guan, H. Huang, Y. Su, H. Huang, N. Wong, and H. Yu, “Aptq: Attention-aware post-training mixed-precision quantization for large language models,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC ’24. ACM, Jun. 2024, p. 1–6. [Online]. Available: <http://dx.doi.org/10.1145/3649329.3658498>
 - [25] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015. [Online]. Available: <https://arxiv.org/abs/1503.02531>
 - [26] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “TinyBERT: Distilling BERT for natural language understanding,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 4163–4174. [Online]. Available: <https://aclanthology.org/2020.findings-emnlp.372/>
 - [27] H. Wang, S. Hassan, Y. Liu, C. Ma, Y. Chen, Q. Li, J. Geng, B. Wang, Y. Tian, Y. Xie, J. Avery, L. Hull, I. Reid, M. Yaqub, and G. Carneiro, “Meta-learned modality-weighted knowledge distillation for robust multi-modal learning with missing data,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.07155>
 - [28] N. Dhar, B. Deng, M. R. Islam, K. F. Ahmad Nasif, L. Zhao, and K. Suo, “Activation sparsity opportunities for compressing general large language models,” in *2024 IEEE International Performance, Computing, and Communications Conference (IPCCC)*. IEEE, Nov. 2024, p. 1–9. [Online]. Available: <http://dx.doi.org/10.1109/IPCCC59868.2024.10850382>
 - [29] Y. Luo, C. Song, X. Han, Y. Chen, C. Xiao, X. Meng, L. Deng, J. Wei, Z. Liu, and M. Sun, “Sparsing law: Towards large language models with greater activation sparsity,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.02335>
 - [30] J. Liu, P. Ponnusamy, T. Cai, H. Guo, Y. Kim, and B. Athiwaratkun, “Training-free activation sparsity in large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2408.14690>
 - [31] Z. Zhang, Z. Liu, Y. Tian, H. Khaitan, Z. Wang, and S. Li, “R-sparse: Rank-aware activation sparsity for efficient llm inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.19449>
 - [32] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning both weights and connections for efficient neural networks,” 2015. [Online]. Available: <https://arxiv.org/abs/1506.02626>
 - [33] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.11695>
 - [34] B. Hassibi and D. Stork, “Second order derivatives for network pruning: Optimal brain surgeon,” in *Advances in Neural Information Processing Systems*, S. Hanson, J. Cowan, and C. Giles, Eds., vol. 5. Morgan-Kaufmann, 1992. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1992/file/303ed4c69846ab36c2904d3ba8573050-Paper.pdf
 - [35] E. Frantar and D. Alistarh, “Sparségpt: massive language models can be accurately pruned in one-shot,” in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML’23. JMLR.org, 2023.
 - [36] T. F. A. van der Ouderaa, M. Nagel, M. V. Baalen, and T. Blankevoort, “The LLM surgeon,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=DYIIRgwg2i>
-

- [37] A. Mishra, J. A. Latorre, J. Pool, D. Stosic, D. Stosic, G. Venkatesh, C. Yu, and P. Micikevicius, “Accelerating sparse deep neural networks,” 2021. [Online]. Available: <https://arxiv.org/abs/2104.08378>
- [38] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, “Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, A. Korhonen, D. Traum, and L. Màrquez, Eds. Florence, Italy: Association for Computational Linguistics, Jul. 2019, pp. 5797–5808. [Online]. Available: <https://aclanthology.org/P19-1580/>
- [39] P. Michel, O. Levy, and G. Neubig, “Are sixteen heads really better than one?” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/2c601ad9d2ff9bc8b282670cdd54f69f-Paper.pdf
- [40] J. Guo, X. Chen, Y. Tang, and Y. Wang, “SlimLLM: Accurate structured pruning for large language models,” in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=2xjUkU7FDdb>
- [41] Y.-C. Hsu, T. Hua, S. Chang, Q. Lou, Y. Shen, and H. Jin, “Language model compression with weighted low-rank factorization,” 2022. [Online]. Available: <https://arxiv.org/abs/2207.00112>
- [42] Z. Yuan, Y. Shang, Y. Song, D. Yang, Q. Wu, Y. Yan, and G. Sun, “Asvd: Activation-aware singular value decomposition for compressing large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2312.05821>
- [43] X. Wang, Y. Zheng, Z. Wan, and M. Zhang, “Svd-llm: Truncation-aware singular value decomposition for large language model compression,” 2025. [Online]. Available: <https://arxiv.org/abs/2403.07378>
- [44] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, and H. Chen, “Streamlining redundant layers to compress large language models,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=IC5RJvRoMp>
- [45] Y. Chen, B. Cheng, J. Han, Y. Zhang, Y. Li, and S. Zhang, “DLP: Dynamic layerwise pruning in large language models,” in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=11id5ppGZ8>
- [46] J. Wee, M. Park, and J. Lee, “Prompt-based depth pruning of large language models,” in *Forty-second International Conference on Machine Learning*, 2025. [Online]. Available: <https://openreview.net/forum?id=hRxHF1xPYB>
- [47] G. Yang, Y. Zhou, X. Zhang, W. Cheng, K. Liu, X. Chen, T. Y. Zhuo, and T. Chen, “Less is more: Towards green code large language models via unified structural pruning,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.15921>
- [48] D. Pisinger and P. Toth, “Knapsack problems,” in *Handbook of Combinatorial Optimization: Volume 1–3*. Springer, 1998, pp. 299–428.
- [49] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, “Qwen2.5 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [50] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann, J. R. Lee, Y. T. Lee, Y. Li, W. Liu, C. C. T. Mendes, A. Nguyen, E. Price, G. de Rosa, O. Saarikivi, A. Salim, S. Shah, X. Wang, R. Ward, Y. Wu, D. Yu, C. Zhang, and Y. Zhang, “Phi-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.08905>
- [51] F. Sandri, E. Cunegatti, and G. Iacca, “2SSP: A two-stage framework for structured pruning of LLMs,” *Transactions on Machine Learning Research*, 2025. [Online]. Available: <https://openreview.net/forum?id=Qd7LzJBg21>
- [52] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.07843>

- [53] D. Paperno, G. Kruszewski, A. Lazaridou, N. Q. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández, “The LAMBADA dataset: Word prediction requiring a broad discourse context,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1525–1534. [Online]. Available: <https://aclanthology.org/P16-1144/>
- [54] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, “Piqa: Reasoning about physical commonsense in natural language,” in *Thirty-Fourth AAAI Conference on Artificial Intelligence*, 2020.
- [55] S. Aroca-Ouellette, C. Paik, A. Roncone, and K. Kann, “PROST: Physical reasoning about objects through space and time,” in *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, C. Zong, F. Xia, W. Li, and R. Navigli, Eds. Online: Association for Computational Linguistics, Aug. 2021, pp. 4597–4608. [Online]. Available: <https://aclanthology.org/2021.findings-acl.404/>
- [56] A. Talmor, J. Herzig, N. Lourie, and J. Berant, “CommonsenseQA: A question answering challenge targeting commonsense knowledge,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4149–4158. [Online]. Available: <https://aclanthology.org/N19-1421>
- [57] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.05457>
- [58] A. Amini, S. Gabriel, S. Lin, R. Koncel-Kedziorski, Y. Choi, and H. Hajishirzi, “MathQA: Towards interpretable math word problem solving with operation-based formalisms,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 2357–2367. [Online]. Available: <https://aclanthology.org/N19-1245>
- [59] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, E. Riloff, D. Chiang, J. Hockenmaier, and J. Tsujii, Eds. Brussels, Belgium: Association for Computational Linguistics, Oct.-Nov. 2018, pp. 2381–2391. [Online]. Available: <https://aclanthology.org/D18-1260/>
- [60] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, “What disease does this patient have? a large-scale open domain question answering dataset from medical exams,” *Applied Sciences*, vol. 11, no. 14, 2021. [Online]. Available: <https://www.mdpi.com/2076-3417/11/14/6421>
- [61] A. Warstadt, A. Parrish, H. Liu, A. Mohananey, W. Peng, S.-F. Wang, and S. R. Bowman, “Blimp: The benchmark of linguistic minimal pairs for english,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 377–392, 2020. [Online]. Available: https://doi.org/10.1162/tacl_a_00321
- [62] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova, “BoolQ: Exploring the surprising difficulty of natural yes/no questions,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 2924–2936. [Online]. Available: <https://aclanthology.org/N19-1300/>
- [63] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [64] S. Reddy, D. Chen, and C. D. Manning, “CoQA: A conversational question answering challenge,” *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 249–266, 2019. [Online]. Available: <https://aclanthology.org/Q19-1016/>
- [65] S. Lin, J. Hilton, and O. Evans, “TruthfulQA: Measuring how models mimic human falsehoods,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, S. Muresan, P. Nakov, and A. Villavicencio, Eds. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 3214–3252. [Online]. Available: <https://aclanthology.org/2022.acl-long.229/>

- [66] R. Rudinger, J. Naradowsky, B. Leonard, and B. Van Durme, “Gender bias in coreference resolution,” in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, M. Walker, H. Ji, and A. Stent, Eds. New Orleans, Louisiana: Association for Computational Linguistics, Jun. 2018, pp. 8–14. [Online]. Available: <https://aclanthology.org/N18-2002/>
- [67] D. Emelin, R. Le Bras, J. D. Hwang, M. Forbes, and Y. Choi, “Moral stories: Situated reasoning about norms, intents, actions, and their consequences,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 698–718. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.54/>
- [68] W. Lian, G. Wang, B. Goodson, E. Pentland, A. Cook, C. Vong, and ”Teknium”, “Slimorca: An open dataset of gpt-4 augmented flan reasoning traces, with verification,” 2023. [Online]. Available: <https://huggingface.co/datasets/Open-Orca/SlimOrca>
- [69] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang, and T. B. Hashimoto, “Stanford alpaca: An instruction-following llama model,” https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [70] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.
- [71] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf
- [72] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Q. Liu and D. Schlangen, Eds. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: <https://aclanthology.org/2020.emnlp-demos.6/>
- [73] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” 07 2024. [Online]. Available: <https://zenodo.org/records/12608602>
- [74] S. Mukherjee, A. Mitra, G. Jawahar, S. Agarwal, H. Palangi, and A. Awadallah, “Orca: Progressive learning from complex explanation traces of gpt-4,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.02707>
- [75] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022. [Online]. Available: <https://openreview.net/forum?id=nZeVKeeFYf9>

Structure of the Appendix

A Implementation Details	21
B Description of Evaluation Tasks	21
C Detailed Results	22
C.1 Comparison of CRAFT Across Methods	22
C.2 Task-Wise Performance	23
C.3 Sensitivity to Calibration Data	24
C.4 Transferability of Importance Scores	24

I A Implementation Details

We implement SNIPER using PyTorch [71] and the HuggingFace Transformers library [72]. All experiments are conducted on a single NVIDIA A100 GPU with 80GB of VRAM and evaluated using the Language Model Evaluation Harness [73]. For importance estimation (Section 3.3), we use a calibration set of 50 samples randomly drawn from the SlimOrca dataset [68, 74], with a context length of 512 tokens.

For post-pruning recovery fine-tuning, we train the pruned models on 2,000 SlimOrca samples with a context length of 1,024 tokens. To maintain computational efficiency, we employ Low-Rank Adaptation [75] rather than full fine-tuning. We use a LoRA rank of 64, a scaling factor of 16, and a dropout probability of 0.05. All models are fine-tuned for a single epoch with a batch size of 2 and a learning rate of 2×10^{-4} .

I B Description of Evaluation Tasks

We devise a suite of 18 diverse tasks to evaluate different pruning methodologies comprehensively. We provide a description of each benchmark task in this section.

Generative Performance. We compute log-perplexity on the WikiText-2 [52] and LAMBADA [53] datasets to assess generative performance. These are language modeling datasets consisting of unlabeled English texts requiring models to understand local and global context to predict subsequent words.

World Understanding. In order to assess models’ common sense capabilities and general understanding of the physical world, we utilize the PIQA [54], PROST [55], and CommonsenseQA [56] datasets. PIQA contains multiple-choice questions (MCQs) testing a model’s general understanding of the physical world with questions such as whether the process of boiling eggs requires water or not. PROST specifically focuses on evaluating model’s understanding of each of 10 physical reasoning concepts such as direction, mass and height. CommonsenseQA also contains MCQs but extends its scope of evaluation to include different types of commonsense knowledge instead of specifically physical understanding.

Domain-Specific Knowledge. We employ the ARC-Easy and ARC-Challenge [57] tasks which contain grade-school level multiple-choice science questions and are designed to test a model’s knowledge of basic science. As their names suggest, the former is an easier version of the task than the latter. We evaluate mathematical capabilities using the MathQA [58] task which contains multiple-choice mathematical word problems. Additionally, we employ the MedQA [60] task which is a challenging dataset containing multiple-choice medical questions from professional medical exams. Lastly, we evaluate a model’s ability to combine its commonsense, domain knowledge and text comprehension capabilities by testing it on the OpenbookQA benchmark [59] which contains complex reasoning questions that require models to refer to a given corpus of salient facts to answer correctly.

Natural Language Understanding and Inference. We gauge a model’s semantic understanding on natural understanding with the help of the BLIMP [61], BoolQ [62], LAMBADA [53], Winogrande [63], and CoQA [64] benchmarks. BLIMP gauges grammatical understanding by making models choose between pairs of sentences that differ in morphology, semantics, or syntax. On the other hand, BoolQ contains binary True/False-style questions based on a reference text, thereby testing a model’s comprehension abilities. While we employ LAMBADA to assess generative performance by computing log-perplexity over it,

we employ its next word prediction format that offers provides models with an incomplete passage and assesses the accuracy with which they are able to predict the next word in the passage by understanding long context. Winogrande tests semantic understanding by providing fill-in-the-blanks questions with binary options, requiring models to choose the most appropriate option given the context. CoQA contains questions based on a multi-turn conversation between two actors, thereby evaluating a model’s ability to understand the nuances of long conversations and how they differ from regular text passages.

Safety, Bias & and Ethics. We assess a pruned model’s alignment to safety and ethical principles by evaluating it on the challenging Winogender [66], TruthfulQA [65], and Moral Stories [67] tasks. Winogender assess systemic bias by offering pairs of highly similar texts that differ only by the gender of a pronoun (e.g., “he” changed to “she”). TruthfulQA poses questions to models which humans may answer incorrectly due to personal biases or misconceptions, assessing the ability of a model to identify and ignore these biases to answer questions truthfully. Lastly, Moral Stories is a dataset containing narratives describing moral and immoral decisions taken by various actors with the goal of assessing whether LLMs are able to discriminate between moral and immoral actions, thereby gauging their alignment with socially acceptable norms.

I C Detailed Results

C.1 Comparison of CRAFT Across Methods

Method	Target CR (%) = r_t	Observed CR (%) = r_o	$\Delta = r_t - r_o$ (↓)	Avg. CRAFT = $\frac{r_o}{r_t}$ (↑)
ReplaceMe SLEB ShortGPT	25	21.19	3.81	0.84
	35	28.23	6.77	
	50	42.38	7.62	
	60	51.79	8.21	
	75	63.57	11.43	
SliceGPT	25	14.60	10.40	0.79
	35	25.99	9.01	
	50	42.13	7.87	
	60	52.82	7.18	
	75	67.86	7.14	
LLM-Pruner	25	17.63	7.37	0.58
	35	19.33	15.67	
	50	27.66	22.34	
	60	33.16	26.84	
	75	41.49	33.51	
2SSP	25	21.19	3.81	0.85
	35	29.64	5.36	
	50	42.38	7.62	
	60	50.87	9.13	
	75	63.57	11.43	
SNIPER	25	24.62	0.38	0.97
	35	33.97	1.03	
	50	48.64	1.36	
	60	58.71	1.29	
	75	71.24	3.76	

Table 8. Quantitative analysis of the *Compression Adherence Factor* (CRAFT) for each method over a diverse range of target compression ratios (25%, 35%, 50%, 60%, 75%).

As depicted in Figure 1, we observe significant disparities between the target and achieved compression ratios for the an-

alyzed baselines, serving as the primary motivator for the dual-stage mixed-granularity design of SNIPER. We provide the quantitative results for this key observation in Table 8. As is evident, baselines exhibit poor adherence to the target compression ratio with CRAFTs going as low as 0.58, thereby reducing reliability and trust. On the contrary, we demonstrate that SNIPER adheres tightly to the target CRs, making it much more reliable in practical settings. Note that since ReplaceMe, SLEB, and ShortGPT are depth pruning baselines, they prune a fixed number of layers for a given compression ratio, thereby leading to the same Δ and CRAFT.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
25%	Base	2.16	1.22	80.96	41.28	77.40	43.60	39.60	79.67	55.03	63.47
	ReplaceMe	4.47	5.36	68.23	35.96	73.71	33.20	27.07	53.45	37.63	52.55
	SliceGPT	4.51	5.72	55.39	29.85	37.10	25.80	23.52	35.61	24.83	24.59
	LLM-Pruner	3.50	4.43	68.44	31.46	21.79	29.60	24.99	52.57	29.18	24.35
	SLEB	3.44	3.17	70.89	29.33	26.37	34.80	26.40	60.35	33.45	30.32
	ShortGPT	10.75	15.65	64.36	33.89	47.83	29.20	25.06	42.97	33.36	27.81
	2SSP	3.27	2.27	68.66	30.28	49.06	33.00	27.71	57.74	35.92	28.99
	SNIPER	3.42	2.89	71.22	31.80	52.42	36.40	26.40	59.30	37.37	35.51
35%	ReplaceMe	6.04	5.68	60.99	34.27	19.98	29.80	23.75	41.84	23.21	26.47
	SliceGPT	5.19	7.70	54.08	30.75	30.47	27.00	21.84	31.82	22.44	25.30
	LLM-Pruner	4.27	5.61	62.02	28.48	21.05	28.00	25.13	43.22	24.83	26.87
	SLEB	4.13	4.81	67.36	29.77	20.48	29.20	24.05	47.39	27.82	26.47
	ShortGPT	11.85	14.43	59.58	31.78	30.96	27.00	22.25	34.26	30.12	31.34
	2SSP	3.97	3.57	60.45	28.64	22.77	26.60	25.33	40.99	25.26	28.75
	SNIPER	4.41	5.00	65.83	29.55	28.09	28.60	24.32	46.84	33.11	28.04

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
25%	Base	83.42	83.91	73.08	73.40	78.80	62.50	54.00	50.17	-	-
	ReplaceMe	74.97	79.76	14.17	66.14	39.80	57.64	55.20	54.01	74.86	20.40
	SliceGPT	75.45	59.45	18.44	50.59	64.80	49.44	45.91	55.33	62.06	20.20
	LLM-Pruner	80.15	63.95	21.54	53.20	30.40	53.47	45.05	57.09	64.56	22.40
	SLEB	80.26	57.00	37.57	58.48	20.60	53.47	44.60	54.98	67.91	20.30
	ShortGPT	69.01	37.83	2.83	58.25	4.60	49.44	51.88	51.71	57.50	26.10
	2SSP	82.92	68.26	55.29	62.12	72.10	55.42	42.08	55.29	76.65	14.00
	SNIPER	80.30	65.08	47.27	65.51	66.60	57.78	47.19	54.10	77.03	12.70
35%	ReplaceMe	76.94	37.89	13.16	51.86	0.60	50.42	48.76	54.53	56.25	27.10
	SliceGPT	72.83	45.38	8.37	50.12	36.90	50.83	46.25	55.50	56.74	22.50
	LLM-Pruner	78.69	58.75	13.76	49.88	21.90	50.56	46.61	56.83	59.66	23.70
	SLEB	75.99	61.56	21.46	52.72	8.90	52.78	44.54	55.29	60.72	24.10
	ShortGPT	61.67	59.30	1.32	57.70	2.60	55.83	52.16	57.10	56.14	28.00
	2SSP	82.54	56.15	36.50	53.99	48.60	52.92	42.79	55.78	64.21	19.10
	SNIPER	76.98	62.63	21.25	60.46	46.30	52.50	46.69	54.69	64.96	19.50

Table 9. Task-specific pruning results for different pruning methods on Llama-3.1-8B-Instruct across various compression ratios (CR) without RFT.

C.2 Task-Wise Performance

Tables 9, 10, 11, and 12 provide the detailed task-wise results for each method on Llama-3.1-8B-Instruct and Qwen3-8B with and without RFT at compression ratios of 25% and 35%. SNIPER demonstrates noteworthy superiority over its baselines across the different testing configurations, repeatedly outperforming them on average and exhibiting the highest per-task consistency among them. Even in the instance where SNIPER lags slightly behind a baseline in average performance (Table 10, 25% CR), it still retains the lowest per-task standard deviation of the group, providing a strong testimony to its excellent robustness to distributional bias.

We also provide the task-wise results for Phi-4 and GPT-OSS-20B in Tables 13 and 14, respectively, where SNIPER surpasses ShortGPT and 2SSP on Avg RP by notable margins. In addition to delivering excellent performance, SNIPER also maintains a considerably lower Std RP compared to them, indicating its impressive ability to adapt to unique architectures.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
-	Base	2.16	1.22	80.96	41.28	77.40	43.60	39.60	79.67	55.03	63.47
-	ReplaceMe	2.92	1.80	70.73	33.61	70.60	34.40	27.94	59.34	39.59	55.46
-	SliceGPT	3.29	3.91	61.37	32.53	53.40	26.40	23.89	44.66	30.97	24.51
-	LLM-Pruner	2.79	2.29	72.63	29.52	30.96	34.20	27.14	58.92	36.60	24.75
-	SLEB	2.72	1.79	73.56	24.79	44.23	37.60	29.62	60.56	36.86	29.30
-	ShortGPT	2.95	1.98	70.62	32.93	74.12	37.40	26.94	56.65	38.65	52.47
-	2SSP	2.64	1.63	73.56	29.18	59.95	34.60	31.36	62.54	40.19	30.09
-	SNIPER	2.85	1.72	70.40	34.73	68.55	35.60	28.51	58.17	40.19	46.11
25%	ReplaceMe	3.40	2.98	68.72	30.39	20.31	28.60	24.29	45.83	26.28	23.10
25%	SliceGPT	3.65	5.37	58.98	29.48	45.86	27.20	22.78	39.94	27.30	24.82
25%	LLM-Pruner	3.07	2.81	67.41	25.13	22.93	32.10	26.43	51.05	31.06	25.22
25%	SLEB	3.05	2.36	69.37	27.21	25.14	31.80	25.93	52.82	31.49	22.70
25%	ShortGPT	3.34	2.54	66.38	29.75	34.81	29.20	24.26	47.77	32.94	28.36
25%	2SSP	2.90	2.00	69.70	26.48	46.68	30.40	28.88	55.43	34.90	28.67
25%	SNIPER	3.09	2.16	68.77	26.92	49.88	29.40	26.30	51.31	32.08	34.56

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
-	Base	83.42	83.91	73.08	73.40	78.80	62.50	54.00	50.17	-	-
-	ReplaceMe	82.67	82.66	64.93	70.64	78.10	57.64	53.74	52.99	88.90	10.50
-	SliceGPT	79.84	71.96	37.96	55.88	66.80	49.58	44.43	54.48	72.49	17.30
-	LLM-Pruner	83.66	55.14	51.80	59.67	61.90	54.17	44.14	54.68	75.11	17.70
-	SLEB	84.02	48.87	61.32	63.69	72.00	54.86	41.88	55.48	78.45	16.80
-	ShortGPT	81.75	83.21	62.49	70.09	76.30	59.86	52.90	53.26	88.47	11.20
-	2SSP	85.42	63.27	64.68	65.67	78.90	57.50	43.45	53.31	83.19	13.90
-	SNIPER	81.89	80.85	66.16	67.88	75.30	58.89	50.60	52.42	87.34	10.00
35%	ReplaceMe	82.47	57.13	36.66	51.70	24.80	51.53	41.93	55.28	65.16	22.60
35%	SliceGPT	78.30	64.37	29.05	54.78	49.60	51.81	48.21	53.68	68.08	18.60
35%	LLM-Pruner	83.53	55.14	42.67	58.17	53.00	52.78	43.38	54.69	69.93	19.40
35%	SLEB	82.75	41.96	51.60	57.46	55.60	55.00	43.35	55.52	70.58	20.10
35%	ShortGPT	79.03	77.46	53.66	67.72	71.70	60.14	49.75	54.12	76.98	18.70
35%	2SSP	84.86	61.50	57.73	62.43	73.60	56.11	39.94	54.51	77.26	15.80
35%	SNIPER	82.43	69.33	57.46	62.51	69.80	55.97	41.87	55.01	77.26	14.80

Table 10. Task-specific pruning results for different pruning methods on Llama-3.1-8B-Instruct across various compression ratios (CR) with RFT.

C.3 Sensitivity to Calibration Data

We analyze the effect of varying calibration-related factors, namely, dataset size and data distribution, on each method’s performance. We provide these results in Tables 15 and 16, respectively. In line with the analysis presented in Section 6, SNIPER demonstrates excellent robustness at varying calibration configurations where existing methods tend to deteriorate either in terms of average performance or per-task consistency.

C.4 Transferability of Importance Scores

SNIPER demonstrates impressive flexibility by maintaining excellent performance even when utilizing heuristics that were not computed for the target compression ratio, as evident by the detailed results in Table 17. This characteristic enhances SNIPER’s practical utility by offering an avenue for users to compute these heuristics once and re-use them for varying target compression ratios.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
-	Base	2.50	1.52	77.53	42.90	78.54	41.60	49.75	81.06	56.23	64.34
-	ReplaceMe	4.11	4.64	62.30	28.13	21.79	31.60	23.15	42.59	25.51	28.20
-	SliceGPT	13.03	18.41	51.47	29.60	20.97	25.60	19.43	26.31	26.20	25.69
25%	LLM-Pruner	3.32	3.09	64.26	28.46	53.48	28.80	28.41	46.59	25.68	29.77
25%	SLEB	3.84	3.66	66.76	28.72	33.84	31.80	25.80	59.72	37.17	30.17
25%	ShortGPT	6.73	11.23	62.08	29.55	39.64	29.60	21.94	41.92	31.14	40.85
25%	2SSP	5.12	7.62	55.50	30.68	18.18	27.80	21.11	32.41	23.29	26.94
25%	SNIPER	3.55	3.75	63.66	30.04	58.72	30.60	26.57	52.44	34.81	36.61
35%	ReplaceMe	6.12	5.87	63.06	30.67	19.17	32.00	23.05	48.49	29.35	28.59
35%	SliceGPT	13.42	18.85	51.52	29.55	19.41	26.00	20.24	26.52	24.92	26.24
35%	LLM-Pruner	4.55	5.95	56.45	28.42	32.84	28.00	22.21	32.41	23.12	26.63
35%	SLEB	4.90	4.99	60.94	32.75	21.70	29.60	23.75	49.50	28.24	28.67
35%	ShortGPT	8.98	12.36	58.98	30.19	47.50	32.00	22.71	33.33	29.44	37.31
35%	2SSP	5.63	9.21	52.61	32.42	19.08	26.00	21.94	30.22	23.72	27.73
35%	SNIPER	4.45	5.77	59.68	32.15	40.30	27.20	22.88	42.72	27.05	32.29

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
-	Base	80.72	86.54	64.10	67.72	81.90	62.22	54.39	51.08	-	-
-	ReplaceMe	78.15	40.31	24.65	51.38	12.70	49.44	45.44	56.53	59.92	23.80
-	SliceGPT	57.66	40.43	7.80	48.15	1.00	49.17	48.50	55.35	48.68	27.50
25%	LLM-Pruner	79.78	72.39	41.39	55.17	70.70	54.72	44.77	53.48	72.64	15.90
25%	SLEB	74.04	66.48	30.04	54.93	18.70	53.06	45.73	55.67	67.53	20.20
25%	ShortGPT	70.86	68.10	10.77	57.06	28.10	58.75	49.17	57.13	63.03	22.90
25%	2SSP	61.04	41.44	11.97	49.96	35.50	50.56	50.61	55.41	56.09	23.10
25%	SNIPER	74.03	69.57	33.34	57.54	70.70	59.58	50.22	54.90	74.41	15.00
35%	ReplaceMe	80.35	48.23	17.45	51.46	6.40	49.86	47.80	56.76	59.40	25.90
35%	SliceGPT	55.95	40.67	5.80	49.88	1.10	47.50	49.78	55.68	48.67	27.70
35%	LLM-Pruner	76.43	61.90	13.12	52.49	46.30	51.67	44.15	56.44	60.63	21.90
35%	SLEB	70.73	50.25	19.91	50.28	3.80	50.69	46.63	57.33	59.67	24.70
35%	ShortGPT	67.94	62.17	13.60	53.51	6.70	55.56	49.73	56.78	58.69	27.10
35%	2SSP	59.32	42.69	6.60	50.36	24.20	49.03	49.35	56.35	54.15	24.40
35%	SNIPER	70.96	58.59	14.38	53.59	45.90	53.19	48.43	56.65	63.71	20.10

Table 11. Task-specific pruning results for different pruning methods on Qwen3-8B across various compression ratios (CR) without RFT.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
-	Base	2.50	1.52	77.53	42.90	78.54	41.60	49.75	81.06	56.23	64.34
25%	ReplaceMe	2.93	2.60	70.46	27.00	34.48	36.20	27.77	57.03	34.73	28.44
	SliceGPT	8.59	12.92	51.20	32.42	19.82	25.80	20.07	27.65	25.09	26.63
	LLM-Pruner	2.83	2.09	70.02	28.14	62.16	34.00	31.66	55.81	33.96	30.95
	SLEB	2.98	2.24	69.80	30.39	51.02	34.40	29.08	55.85	37.37	35.19
	ShortGPT	3.44	2.85	67.03	32.18	68.39	30.20	25.90	52.36	33.96	54.28
	2SSP	2.82	2.04	69.75	32.21	68.96	32.20	26.73	61.36	38.14	35.98
	SNIPER	3.04	2.41	68.01	32.29	67.49	31.40	31.09	62.88	38.99	42.42
35%	ReplaceMe	4.15	4.25	61.86	29.81	22.69	28.40	22.14	39.94	27.73	26.08
	SliceGPT	8.52	13.13	51.36	32.16	19.66	24.60	19.46	26.98	24.06	27.81
	LLM-Pruner	3.30	3.63	63.87	27.82	44.06	29.00	23.45	51.85	28.75	26.24
	SLEB	3.38	2.83	64.91	30.82	39.72	32.00	25.93	52.82	32.68	28.59
	ShortGPT	3.80	4.17	64.04	31.85	54.87	31.00	23.75	45.96	29.95	43.44
	2SSP	3.52	3.59	64.04	28.23	46.85	29.20	23.45	47.14	30.29	29.62
	SNIPER	3.36	3.06	65.29	31.45	55.69	29.20	26.20	53.66	31.66	36.53

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
-	Base	80.72	86.54	64.10	67.72	81.90	62.22	54.39	51.08	-	-
25%	ReplaceMe	82.84	41.10	42.81	56.43	54.40	52.36	44.35	55.28	72.30	18.60
	SliceGPT	57.22	38.47	0.00	50.91	1.10	47.22	49.76	57.07	50.07	30.00
	LLM-Pruner	82.69	59.02	55.02	62.12	76.40	55.28	47.07	54.25	80.11	15.00
	SLEB	79.76	60.40	54.05	58.71	69.60	56.25	48.93	54.18	78.89	13.80
	ShortGPT	79.28	80.06	47.70	63.46	69.40	59.31	48.38	54.73	80.21	15.20
	2SSP	82.76	63.39	57.99	64.80	77.80	57.50	49.79	52.11	82.77	13.90
	SNIPER	80.85	77.19	51.12	63.22	74.20	58.61	47.99	54.51	82.64	12.20
35%	ReplaceMe	76.07	53.27	32.82	54.78	44.40	56.67	44.30	56.82	64.00	21.60
	SliceGPT	58.83	38.14	0.00	51.38	1.00	50.69	49.18	57.44	50.17	30.40
	LLM-Pruner	81.68	48.84	31.36	53.51	59.70	53.33	44.85	55.98	68.34	19.10
	SLEB	79.03	43.98	44.17	54.54	46.10	53.61	48.66	55.02	70.47	17.50
	ShortGPT	77.52	68.01	32.02	58.96	61.20	60.00	43.61	55.73	72.27	18.30
	2SSP	73.65	53.82	35.15	56.20	64.60	53.33	46.23	55.47	69.23	17.50
	SNIPER	81.71	72.20	41.51	57.93	63.40	55.00	47.49	55.95	75.13	15.90

Table 12. Task-specific pruning results for different pruning methods on Qwen3-8B across various compression ratios (CR) with RFT.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
-	Base	2.05	1.25	81.34	54.46	74.53	45.20	47.07	72.81	56.06	62.53
35%	ShortGPT	3.38	3.32	68.83	54.65	39.07	36.40	30.55	60.14	41.55	31.58
	2SSP	2.77	3.38	65.51	34.81	42.51	28.00	24.99	50.46	32.34	30.95
	SNIPER	2.77	2.33	73.94	34.00	66.26	37.60	32.63	66.08	42.58	41.79

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
-	Base	81.69	86.18	72.31	76.56	82.70	71.67	59.34	47.38	-	-
35%	ShortGPT	78.28	65.05	41.06	71.67	70.90	68.61	49.89	53.83	77.18	16.60
	2SSP	80.41	60.55	42.97	54.85	74.50	52.22	48.24	54.39	70.16	16.20
	SNIPER	82.45	75.44	50.51	63.38	78.70	62.50	44.65	53.95	81.58	12.60

Table 13. Task-specific pruning results for different pruning methods on Phi-4 for a CR of 35% with RFT.

CR	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambda (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
-	Base	2.59	1.86	78.62	45.28	73.06	40.60	47.84	77.53	51.88	62.53
-	ShortGPT	3.17	2.95	69.64	30.56	38.33	32.00	33.87	60.40	33.79	30.09
35%	2SSP	3.26	2.73	73.12	33.43	61.92	35.20	31.99	65.28	40.27	39.91
	SNIPER	3.04	2.38	72.96	32.39	61.51	37.00	35.75	66.16	42.41	45.09

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambda (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
-	Base	81.00	86.42	60.76	67.88	80.80	62.22	59.13	53.13	-	-
-	ShortGPT	77.70	54.43	40.07	53.51	0.10	53.61	46.73	56.38	70.51	22.10
35%	2SSP	80.60	75.11	46.87	63.30	79.10	61.53	53.83	53.79	84.68	11.40
	SNIPER	80.44	79.82	52.05	61.72	75.70	58.89	51.79	55.72	86.99	8.80

Table 14. Task-specific pruning results for different pruning methods on GPT-OSS-20B for a CR of 35% with RFT.

# Calibration Samples	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambda (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
50	ReplaceMe	2.93	2.60	70.46	27.00	34.48	36.20	27.77	57.03	34.73	28.44
	SliceGPT	8.59	12.92	51.20	32.42	19.82	25.80	20.07	27.65	25.09	26.63
	LLM-Pruner	2.83	2.09	70.02	28.14	62.16	34.00	31.66	55.81	33.96	30.95
	SLEB	2.98	2.24	69.80	30.39	51.02	34.40	29.08	55.85	37.37	35.19
	ShortGPT	3.44	2.85	67.03	32.18	68.39	30.20	25.90	52.36	33.96	54.28
	2SSP	2.82	2.04	69.75	32.21	68.96	32.20	26.73	61.36	38.14	35.98
250	SNIPER	3.04	2.41	68.01	32.29	67.49	31.40	31.09	62.88	38.99	42.42
	ReplaceMe	2.91	2.54	71.00	27.04	33.99	36.00	27.97	57.74	36.52	28.12
	SliceGPT	8.39	13.10	50.11	34.69	19.66	28.00	20.30	27.69	24.40	27.34
	LLM-Pruner	2.86	2.29	70.24	30.17	60.61	32.80	31.66	63.34	38.06	30.09
	SLEB	3.00	2.55	72.04	28.57	31.12	39.00	29.01	59.51	37.12	29.22
	ShortGPT	3.41	2.79	66.81	34.26	72.97	29.80	26.16	52.27	35.50	56.01
1000	2SSP	3.08	2.95	65.83	32.11	57.08	29.80	24.29	50.84	31.91	31.27
	SNIPER	3.01	2.50	69.59	32.43	61.02	32.80	28.74	62.46	38.48	38.57
	ReplaceMe	2.91	2.54	71.00	27.04	33.99	36.00	27.97	57.74	36.52	28.12
	SliceGPT	7.92	11.98	51.09	28.98	18.76	27.20	20.34	27.65	23.38	26.24
	LLM-Pruner	2.94	2.35	68.55	27.78	59.54	33.60	29.05	60.99	37.88	30.56
	SLEB	2.98	2.58	70.89	29.77	37.35	35.20	29.95	60.19	34.73	31.19
	ShortGPT	3.41	2.78	67.14	33.22	75.43	30.20	25.60	51.05	34.90	58.05
	2SSP	2.97	3.02	67.57	30.68	60.69	29.00	24.26	53.45	32.85	29.93
	SNIPER	3.01	2.49	69.70	31.47	60.20	32.80	28.74	62.16	37.71	38.96

(continued)

# Calibration Samples	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambda (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
50	ReplaceMe	82.84	41.10	42.81	56.43	54.40	52.36	44.35	55.28	72.30	18.60
	SliceGPT	57.22	38.47	0.00	50.91	1.10	47.22	49.76	57.07	50.07	30.00
	LLM-Pruner	82.69	59.02	55.02	62.12	76.40	55.28	47.07	54.25	80.11	15.00
	SLEB	79.76	60.40	54.05	58.71	69.60	56.25	48.93	54.18	78.89	13.80
	ShortGPT	79.28	80.06	47.70	63.46	69.40	59.31	48.38	54.73	80.21	15.20
	2SSP	82.76	63.39	57.99	64.80	77.80	57.50	49.79	52.11	82.77	13.90
250	SNIPER	80.85	77.19	51.12	63.22	74.20	58.61	47.99	54.51	82.64	12.20
	ReplaceMe	83.41	44.04	44.87	56.59	54.20	52.50	44.71	55.03	73.08	18.00
	SliceGPT	58.83	38.38	0.00	50.43	0.60	49.31	49.24	57.73	50.91	28.90
	LLM-Pruner	82.49	42.72	52.20	62.35	77.10	56.11	45.88	53.89	79.26	15.60
	SLEB	81.46	58.87	45.49	57.93	43.10	53.61	42.78	55.74	73.99	18.20
	ShortGPT	78.62	80.83	48.61	62.75	67.50	58.06	48.92	54.72	81.00	13.40
1000	2SSP	74.26	72.17	45.12	57.70	68.70	58.47	45.98	55.44	75.31	15.10
	SNIPER	80.29	76.70	47.70	62.98	73.60	59.17	47.58	53.98	81.28	12.60
	ReplaceMe	83.41	44.04	44.87	56.59	54.20	52.50	44.71	55.03	73.08	18.00
	SliceGPT	60.30	41.28	5.80	50.28	0.80	48.75	49.49	56.71	50.19	28.20
	LLM-Pruner	82.16	46.02	51.56	59.91	75.60	55.00	45.54	54.98	78.02	15.50
	SLEB	81.75	42.60	45.08	55.09	45.30	54.44	43.26	56.09	73.08	17.50
	ShortGPT	78.95	80.64	49.00	63.22	67.50	59.03	48.45	54.78	81.23	14.10
	2SSP	74.77	55.72	44.15	58.49	67.10	51.94	47.81	56.85	74.26	15.40
	SNIPER	80.38	77.98	47.99	63.46	76.70	59.44	48.33	54.01	81.52	13.00

Table 15. Detailed results for the impact of varying number of calibration samples selected from the Slim Orca dataset to identify model components to prune from Qwen3-8B. All results have been obtained after RFT.

Calibration Dataset	Method	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
Slim Orca [68]	ReplaceMe	2.93	2.60	70.46	27.00	34.48	36.20	27.77	57.03	34.73	28.44
	SliceGPT	8.59	12.92	51.20	32.42	19.82	25.80	20.07	27.65	25.09	26.63
	LLM-Pruner	2.83	2.09	70.02	28.14	62.16	34.00	31.66	55.81	33.96	30.95
	SLEB	2.98	2.24	69.80	30.39	51.02	34.40	29.08	55.85	37.37	35.19
	ShortGPT	3.44	2.85	67.03	32.18	68.39	30.20	25.90	52.36	33.96	54.28
	2SSP	2.82	2.04	69.75	32.21	68.96	32.20	26.73	61.36	38.14	35.98
	SNIPER	3.04	2.41	68.01	32.29	67.49	31.40	31.09	62.88	38.99	42.42
Alpaca [69]	ReplaceMe	3.78	4.11	65.78	42.47	40.05	32.60	25.96	52.32	35.41	30.32
	SliceGPT	9.55	13.84	50.82	34.40	19.57	26.40	20.37	27.95	25.60	27.73
	LLM-Pruner	3.00	2.73	68.50	29.68	51.84	32.60	30.52	61.62	36.78	32.05
	SLEB	3.51	3.27	65.45	32.86	41.77	29.40	29.08	52.02	32.59	33.46
	ShortGPT	3.29	2.56	67.57	35.32	72.56	33.80	26.97	55.43	37.20	52.55
	2SSP	2.94	2.49	68.23	31.08	61.02	31.00	28.91	61.07	38.74	36.61
	SNIPER	3.01	2.49	70.19	32.86	61.34	32.60	30.32	61.95	39.33	38.26
C4 [70]	ReplaceMe	3.78	4.11	65.78	42.47	40.05	32.60	25.96	52.32	35.41	30.32
	SliceGPT	9.24	13.91	48.37	34.01	19.57	26.20	20.54	27.69	25.00	27.73
	LLM-Pruner	3.00	2.72	67.41	28.78	53.15	33.80	30.25	61.62	36.35	30.48
	SLEB	3.21	2.57	69.48	36.92	55.61	34.60	31.39	62.00	39.76	33.78
	ShortGPT	3.63	3.68	66.05	43.07	74.45	31.20	26.57	52.65	34.04	51.93
	2SSP	3.25	3.44	62.57	26.57	29.81	28.40	23.99	46.17	29.35	27.34
	SNIPER	3.01	2.49	69.04	33.52	61.75	32.60	29.98	62.46	39.08	39.20

(continued)

CR	Method	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
Slim Orca [68]	ReplaceMe	82.84	41.10	42.81	56.43	54.40	52.36	44.35	55.28	72.30	18.60
	SliceGPT	57.22	38.47	0.00	50.91	1.10	47.22	49.76	57.07	50.07	30.00
	LLM-Pruner	82.69	59.02	55.02	62.12	76.40	55.28	47.07	54.25	80.11	15.00
	SLEB	79.76	60.40	54.05	58.71	69.60	56.25	48.93	54.18	78.89	13.80
	ShortGPT	79.28	80.06	47.70	63.46	69.40	59.31	48.38	54.73	80.21	15.20
	2SSP	82.76	63.39	57.99	64.80	77.80	57.50	49.79	52.11	82.77	13.90
	SNIPER	80.85	77.19	51.12	63.22	74.20	58.61	47.99	54.51	82.64	12.20
Alpaca [69]	ReplaceMe	78.37	83.39	34.16	63.30	68.80	59.31	50.26	54.57	75.66	18.70
	SliceGPT	56.02	37.92	0.00	51.30	0.40	49.86	49.29	57.03	50.45	28.60
	LLM-Pruner	81.05	55.72	42.93	59.20	69.40	54.72	45.48	56.65	76.67	14.40
	SLEB	75.80	71.32	43.97	56.99	59.90	57.64	48.01	55.94	73.78	15.00
	ShortGPT	79.83	81.65	50.69	62.90	68.10	58.75	47.52	54.93	82.59	13.50
	2SSP	82.84	52.75	49.23	62.12	71.80	55.42	48.19	54.91	79.06	14.30
	SNIPER	80.61	77.34	48.34	63.38	73.70	59.72	47.54	54.10	81.79	13.10
C4 [70]	ReplaceMe	78.37	83.39	34.16	63.30	68.80	59.31	50.26	54.57	75.66	18.70
	SliceGPT	53.69	37.83	0.00	49.49	0.80	50.83	54.53	57.28	50.50	29.10
	LLM-Pruner	80.33	57.40	43.06	60.62	70.70	53.19	45.26	57.37	76.73	14.80
	SLEB	78.83	74.68	49.31	62.75	62.20	55.00	52.28	54.48	80.57	12.70
	ShortGPT	78.70	81.25	38.31	65.04	70.20	59.58	49.26	54.58	80.63	15.30
	2SSP	71.82	43.52	37.20	57.06	65.90	51.81	47.60	57.18	67.48	18.80
	SNIPER	80.50	77.46	48.25	63.14	73.70	60.69	47.61	54.04	81.93	13.10

Table 16. Detailed results for impact of calibration data distribution on post-pruning performance of Qwen3-8B at a compression ratio of 25%. Sample count is kept constant at 50. All results have been obtained after RFT.

CR	Transfer Regime	Generative		World Understanding			Domain-Specific				
		Wikitext (Log-PPL ↓)	Lambada (Log-PPL ↓)	PIQA (Acc ↑)	PROST (Acc ↑)	CommonsenseQA (Acc ↑)	OpenbookQA (Acc ↑)	MathQA (Acc ↑)	ARC-Easy (Acc ↑)	ARC-Challenge (Acc ↑)	MedQA (Acc ↑)
25%	25% → 25% (No transfer)	3.55	3.75	63.66	30.04	58.72	30.60	26.57	52.44	34.81	36.61
	35% → 25% (High-to-low)	3.55	3.76	63.44	29.93	58.40	30.60	26.80	52.57	35.07	35.59
	50% → 25% (High-to-low)	3.55	3.71	64.31	29.52	54.63	30.80	26.27	55.72	33.28	32.60
35%	35% → 35% (No transfer)	4.41	5.00	65.83	29.55	28.09	28.60	24.32	46.84	33.11	28.04
	25% → 35% (Low-to-high)	4.38	5.76	60.34	31.49	36.36	25.40	23.18	40.95	26.20	30.72
	50% → 35% (High-to-low)	4.52	5.64	60.12	32.50	40.62	28.80	22.58	41.58	26.45	32.76
50%	50% → 50% (No transfer)	7.74	15.39	55.44	32.61	21.05	28.80	20.17	31.52	25.94	27.57
	25% → 50% (Low-to-high)	7.46	11.65	55.82	31.47	19.25	27.00	19.30	31.86	23.98	27.18
	35% → 50% (Low-to-high)	8.43	14.50	55.99	33.95	20.48	25.40	20.54	32.96	23.98	28.28

(continued)

CR	Transfer Regime	NLU & NLI					Safety, Bias & Ethics			Avg RP (%) (↑)	Std RP (%) (↓)
		BLIMP (Acc ↑)	BoolQ (Acc ↑)	Lambada (Acc ↑)	Winogrande (Acc ↑)	CoQA (Acc ↑)	Winogender (Acc ↑)	TruthfulQA (Acc ↑)	Moral Stories (Acc ↑)		
25%	25% → 25% (No transfer)	74.03	69.57	33.34	57.54	70.70	59.58	50.22	54.90	74.41	15.00
	35% → 25% (High-to-low)	73.90	69.76	33.28	58.09	68.90	59.44	50.19	55.03	74.24	17.00
	50% → 25% (High-to-low)	74.19	74.34	33.34	56.51	69.50	58.75	50.56	54.62	73.92	17.40
35%	35% → 35% (No transfer)	70.96	58.59	14.38	53.59	45.90	53.19	48.43	56.65	63.71	20.10
	25% → 35% (Low-to-high)	71.82	64.95	13.91	53.12	44.80	55.83	48.74	56.68	63.48	22.90
	50% → 35% (High-to-low)	70.95	62.97	15.82	53.67	33.80	58.33	47.72	56.38	63.83	22.90
50%	50% → 50% (No transfer)	60.02	54.99	34.90	51.30	1.50	55.00	48.34	57.19	53.30	28.60
	25% → 50% (Low-to-high)	61.88	38.10	2.72	50.43	0.30	49.58	48.42	57.92	51.46	30.00
	35% → 50% (Low-to-high)	62.77	46.33	1.52	49.01	0.10	53.61	49.78	57.73	52.45	30.90

Table 17. Detailed results for SNIPER under various transfer regimes wherein importance scores computed for a particular compression ratio are utilized to prune a model to a different capacity.